

1 Reminder	1
1.1 Observations and Tricks	1
1.2 Bug List	1
2 Init (Linux)	1
2.1 vimrc	1
2.2 bashrc	1
3 Basic	1
3.1 Template (Using Codebook)	1
3.2 PBDS, Random	1
3.3 Debug	1
3.4 SVG Writer	1
3.5 Python	2
3.6 Stress Tests	2
4 Data Structure	2
4.1 Mo's Algorithm	2
4.2 CDQ	2
4.3 Persistent Treap	3
4.4 Li Chao Tree	3
4.5 Time Segment Tree	3
4.6 Segment Tree Beats	3
5 DP	4
5.1 SOS DP	4
5.2 Divide and Conquer DP	4
5.3 Dynamic DP	4
5.4 Convex Hull Trick	4
6 Graph	4
6.1 Max Clique	4
6.2 Bellman-Ford	5
6.3 System of Difference Constraints	5
6.4 Graph Girth	5
6.5 Euler Trail	5
6.6 Vertex BCC (Round Square Tree)	5
6.7 Edge BCC	6
6.8 Kth Shortest Path	6
6.9 SCC - Tarjan	7
6.10 2SAT	7
7 Tree	7
7.1 Tree Isomorphism (Rooted Trees)	7
7.2 Tree Isomorphism (Unrooted Trees)	7
7.3 Heavy Light Decomposition	7
7.4 Virtual Tree	7
8 Matching	7
8.1 Bipartite Matching	7
8.2 Bipartite Weighted Matching	8
8.3 General Matching	8
9 Flow	8
9.1 Flow Methods	8
9.2 Dinic (Flow Decomposition)	9
9.3 ISAP	9
9.4 Bounded Max Flow	10
9.5 MCMF	10
9.6 Push-Relabel	10
9.7 Gomory-Hu Tree	11
9.8 Global Min Cut	11
10 String	11
10.1 Rolling Hash	11
10.2 KMP	11
10.3 Z Value	11
10.4 Manacher	11
10.5 Suffix Array + LCP	11
10.6 Suffix Automaton	12
10.7 Minimum Rotation	12
10.8 AC Automaton	12
10.9 AC Automaton - Dict Links	12
10.10 AC Automaton - Counting Patterns	12
10.11 AC Automaton - Pattern Position	12
11 Geometry	13
11.1 Template	13
11.2 Basic	13
11.3 Line / Segment Intersections	13

11.4 Point-Line / Segment Distance	13
11.5 Sort	13
11.6 Convex Hull	13
11.7 Triangle Centers	13
11.8 Point Inside	14
11.9 Point In Triangle	14
11.10 Vector In Polygon	14
11.11 Polygon Cut	14
11.12 Half-Plane Intersection	14
11.13 Cyclic Ternary Search	14
11.14 Circle Intersections	14
11.15 Tangents	15
11.16 Bisector	15
11.17 Minimum Enclosing Circle	15
11.18 Minkowski Sum	15
11.19 Convex Hull Distance	15
11.20 Closest Point Pair	15
11.21 Polygon-Circle Intersection	15
11.22 Union of Polygons	15
11.23 Rotating Sweep Line	16
11.24 Pick's Theorem	16
12 Modular	16
12.1 Basic	16
12.2 Mod Sum / Floor Sum	16
12.3 Mod Sqrt	16
12.4 BSGS / exBSGS	16
13 Number Theory	17
13.1 Miller Rabin	17
13.2 Pollard Rho	17
13.3 Linear Sieve	17
13.4 Lucy (prime counting / prime sum)	17
13.5 Black (Min25 sieve)	17
13.6 Fast GCD	17
13.7 CRT	18
13.8 Extended Lucas	18
13.9 Divisor Block	18
13.10 Formulas	18
14 Polynomial	18
14.1 Generating Functions	18
14.2 NTT	19
14.3 FFT	19
14.4 FFT Mod (arbitrary modulus)	19
14.5 FWHT	19
14.6 Polynomial Operations	19
14.7 Berlekamp Massey	20
14.8 Linear Recurrence (Kitamasa)	21
14.9 Lagrange Interpolation (iota points)	21
15 Linear Algebra	21
15.1 Gauss (mod p)	21
15.2 Solve Linear Equations	21
15.3 Gauss (GF(2) / XOR basis)	21
15.4 Solve Linear Equations (GF(2))	21
15.5 Determinant	22
16 Numerical	22
16.1 Integrate (Simpson / adaptive)	22
16.2 Linear Programming	22
16.3 Simplex	22
17 Combinatorics	22
17.1 Catalan Number	22
17.2 Bertrand's Ballot Theorem	23
17.3 Burnside's Lemma	23
17.4 Binomial Identities	23
17.5 Stirling Numbers	23
17.6 Derangement / Inclusion-Exclusion	23
17.7 Pólya (necklaces)	23
18 Special Numbers	23
18.1 Fibonacci Series	23
18.2 Prime Numbers	23
18.3 Number of Divisors	23
18.4 Distinct Prime Factors	23

- DFS 的時候不小心覆寫到全域變數
- 多筆測資不能沒讀完直接 return
- vector 超級肥，小 vector 請用 array，例如矩陣快速冪

2 Init (Linux)

2.1 vimrc

```
1 syn on
2 set nu rnu cul mouse=a cin et ts=2 sw=2 sts=2 autochdir
  cb=unnamedplus
3 no ; :
4 ino {<CR> {<CR>}<Esc>ko
5 ca Hash w !cpp -dD -P \\\ tr -d '[:space:]' \\\ openssl sha256
  -r \\\ cut -c6
```

```
1 " Gino's .vimrc "
2 "no <C-h> ^ | <C-l> $ | h b | l e | b l | e h
```

2.2 bashrc

```
1 gogo() {
2   g++ -std=c++20 -O2 "$1" && echo run && ./a.out
3 }
4 alias vig='vim -u ~/.vimrc-gino'
```

3 Basic

3.1 Template (Using Codebook)

```
1 // CRyptoGRapheR
2 #define SZ(c) ((int)(c).size())
3 // kactl
4 #define sz(x) (int)(x).size()
5 #define rep(i, a, b) for(int i=(a); i<(b); i++)
6 using ll = long long;
7 using vi = vector<int>;
8 using pii = pair<int, int>;
9 // ckiseki
10 #define all(x) begin(x), end(x)
11 #define pb emplace_back
12 #define REP(i, l, r) for (int i=(l); i<=(r); ++i)
13 const ll LINF = 1e18;
14 template<class A, class B> bool chmin(A& a, B& b) {
15   return b < a ? a = b, 1 : 0;
16 }
```

3.2 PBDS, Random

```
1 #include <bits/extc++.h>
2 using namespace __gnu_pbds;
3 // map
4 tree<int, int, less<>, rb_tree_tag,
  tree_order_statistics_node_update> tr;
5 tr.order_of_key(element);
6 tr.find_by_order(rank);
7 // set
8 tree<int, null_type, less<>, rb_tree_tag,
  tree_order_statistics_node_update> tr;
9 tr.order_of_key(element);
10 tr.find_by_order(rank);
11 // priority queue
12 __gnu_pbds::priority_queue<int, less<int>> big_q;
13 __gnu_pbds::priority_queue<int, greater<int>> small_q;
14 ql.join(q2); // join
15
16 mt19937
  gen(chrono::steady_clock::now().time_since_epoch().count());
17 uniform_int_distribution<int>(a, b)(rng) // [a, b]
18 uniform_real_distribution<double>(a, b)(rng) // (a, b)
19 shuffle(v.begin(), v.end(), gen);
```

3.3 Debug

```
1 #define debug(...) cerr << "\x1b[33m" #__VA_ARGS__ " :
  \x1b[0m" \
2   [](auto&&... a){ ((cerr << a << ' '), ...); }
  (__VA_ARGS__) , cerr << '\n'
3 #define output(...) \
4   [](auto&&... a){ ((cout << a << ' '), ...); }
  (__VA_ARGS__) , cout << '\n'
5 int v = 42;
6 format("{:b}", v); // "101010"
7 format("{:#b}", v); // "0b101010"
8 format("{:x}", v); // "0x2a"
9 format("{:X}", v); // "0X2A"
10 int x = 7;
11 format("{:05d}", x); // "00007"
12 format("{:>5}", x); // " 7"
13 format("{:<5}", x); // "7 "
14 format("{:^5}", x); // " 7 "
15 format("{:*5}", x); // "***7***"
16 double pi = 3.1415926535;
17 format("{:.2f}", pi); // "3.14"
```

3.4 SVG Writer

```
1 // Author: ckiseki
2 class SVG {
3   void p(string_view s) { o << s; }
```

1 Reminder

1.1 Observations and Tricks

- Contribution Technique
- 二分圖/Spanning Tree/DFS Tree
- 行、列操作互相獨立
- 奇偶性
- 當 s, t 遞增並且 $t = f(s)$ ，對 s 二分搜不好做，可以改成對 t 二分搜，再算 $f(t)$
- 啟發式合併
- Permutation Normalization (做一些平移對齊兩個 permutation)
- 枚舉 $a_1 \sim a_n$ 再枚舉 $a_n \sim a_1$ 可以包在一個迴圈
- 兩個凸型函數相加還是凸型函數，相減不一定
- 一個區間的 $\text{mex} = k$ ，表示這個區間包含 $0 \sim k-1$ 所有數字，並且「 U -區間」的最小值 $= k$ 。

1.2 Bug List

- 沒開 long long
- 陣列截出界／陣列開不夠大
- 寫好的函式 (例如 `init()`、`build()`) 忘記呼叫
- 0-base / 1-base
- 線段樹改值懶標初始值不能設為 0

```

4 void p(string_view s, auto v, auto... vs) {
5     auto i = s.find('$');
6     o << s.substr(0, i) << v, p(s.substr(i + 1), vs...);
7 }
8 ofstream o; string c = "red";
9 public:
10 SVG(auto f, auto x1, auto y1, auto x2, auto y2) : o(f) {
11     p("<svg xmlns='http://www.w3.org/2000/svg' "
12       "viewBox='$ $ $ $'>\n"
13       "<style>{*stroke-width:0.5%;}</style>\n",
14       x1, -y2, x2 - x1, y2 - y1); }
15 ~SVG() { p("</svg>\n"); }
16 void color(string nc) { c = nc; }
17 void line(auto x1, auto y1, auto x2, auto y2) {
18     p("<line x1='$' y1='$' x2='$' y2='$' stroke='$'>\n",
19       x1, -y1, x2, -y2, c); }
20 void circle(auto x, auto y, auto r) {
21     p("<circle cx='$' cy='$' r='$' stroke='$' "
22       "fill='none'/>\n", x, -y, r, c); }
23 void text(auto x, auto y, string s, int w = 12) {
24     p("<text x='$' y='$' font-size='$px'>$</text>\n",
25       x, -y, w, s); }
26 };

```

```

1 SVG svg("out.svg", -200, -200, 200, 200);
2 svg.color("lightgray"); // ↓ drawing x-y plane
3 svg.line(-200, 0, 200, 0); svg.line(0, -200, 0, 200);
4 svg.color("blue"); svg.line(10, 10, 80, 80);
5 svg.color("red"); svg.circle(50, 50, 1);

```

3.5 Python

b16508

```

1 import sys
2 input = sys.stdin.readline
3 readStr = lambda: input().rstrip('\r\n')
4 readInt = lambda: int(input())
5 readInts = lambda: list(map(int, input().split()))
6 from decimal import *
7 getcontext().prec = 50 # precision
8 x = Decimal(str(x))
9 y = Decimal("6.7")
10 x *= y
11 print(x.quantize(Decimal("0.000001"),
12   rounding=ROUND_HALF_EVEN))
13 # ROUND_CEILING : To INF (2.9 -> 3, -2.1 -> -2)
14 # ROUND_FLOOR : To -INF (2.1 -> 2, -2.9 -> -3)
15 # ROUND_HALF_EVEN : To nearest even (2.5 -> 2, 3.5 -> 4)
16 # ROUND_HALF_UP : Half away from 0 (2.5 -> 3, -2.5 -> -3)
17 # ROUND_HALF_DOWN : Half towards 0 (2.5 -> 2, -2.5 -> -2)
18 # ROUND_UP : Away from 0 (2.1 -> 3, -2.1 -> -3)
19 # ROUND_DOWN : Truncate to 0 (2.9 -> 2, -2.9 -> -2)
20 # <Default>: ROUND_HALF_EVEN
21 # <Standard School Math>: ROUND_HALF_UP

```

3.6 Stress Tests

```

1 from random import *
2 n = randint(1, 10)
3 arr = list(randint(1, 20) for i in range(n))
4 print(*arr)
5 # permutation / non-repetitive sequence
6 perm = sample(range(1, n + 1), n)
7 arr = sample(range(1, 10000 + 1), n)
8 # string
9 from string import *
10 s = "".join(choices(ascii_lowercase, k=n))
11 t = "".join(choices(ascii_lowercase[:3], k=n)) # (a|b|c)*
12 u = "".join(choices(ascii_lowercase + ascii_uppercase +
13   digits, k=n))
14 # palindrome
15 half = "".join(choices(ascii_lowercase, k=(n + 1) // 2))
16 palindrome = half + (half[-2::-1] if n % 2 else half[::-1])
17 # ===== tree =====
18 n = 30
19 ## shallow tree
20 edges = [(randint(1, u - 1), u) for u in range(2, n + 1)]
21 ## deep tree
22 (suggested k: 2 ~ 5 or int(sqrt(n)) or 2n/D (D is
23   expected depth))
24 from math import *
25 k = int(sqrt(n))
26 edges = [(randint(max(1, u - k), u - 1), u) for u in range(2, n
27   + 1)]
28 for e in edges:
29     print(*e)
30 edges = [(1, i) for i in range(2, n + 1)] ## star
31 edges = [(i, i + 1) for i in range(1, n)] ## chain
32 edges = [(i // 2, i) for i in range(2, n + 1)] ## complete
33   binary tree
34 # ===== graph =====
35 ## not necessarily connected, no multi-edge or self-loop
36 n = 10

```

```

33 density = 0.3 # graph density (0.0 ~ 1.0)
34 edges = [(i, j) for i in range(1, n + 1) for j in range(i + 1,
35   n + 1) if random() < density]
36 ## connected, no multi-edge or self-loop
37 n, extra_edges = 10, 5
38 edges = set()
39 ### ... generate spanning tree with codes above
40 while len(edges) < n - 1 + extra_edges:
41     u, v = randint(1, n), randint(1, n)
42     if u != v:
43         edges.add((min(u, v), max(u, v)))
44 edges = list(edges)
45 ## DAG
46 perm = sample(range(1, n + 1), n)
47 edges = []
48 for _ in range(m):
49     i, j = sorted(sample(range(n), 2))
50     edges.append((perm[i], perm[j]))
51 ## Bipartite Graph
52 n1, n2 = 6, 7
53 left, right = list(range(1, n1 + 1)), list(range(n1 + 1,
54   n1 + n2 + 1))
55 edges = [(choice(left), choice(right)) for _ in range(m)]

```

```

1 g++ -std=c++20 -o buggy $1
2 g++ -std=c++20 -o ac ac.cpp
3 for i in {1..100}; do
4     echo "TEST $i"
5     python3 gen.py > in
6     ./ac < in > oa
7     ./buggy < in > ob
8     if ! diff -q oa ob; then
9         cat in <(echo "=== AC") oa <(echo "=== WA") ob
10        break
11    fi
12 done

```

4 Data Structure

4.1 Mo's Algorithm

0e54b1

```

1 // segments are 0-based
2 int B = max(1, (int)sqrt(n));
3 sort(Q, begin(), Q.end(), [&](const auto& a, const auto& b) {
4     if (a[0]/B != b[0]/B) return a[0]<b[0];
5     return (a[0]/B & 1) ? a[1]<b[1] : a[1]>b[1];
6 });
7 ll cur = 0; // current answer
8 int pl = 0, pr = -1;
9 for (auto& qi : Q) {
10    // get (l, r, qid) from qi
11    while (pl < l) del(pl++); while (pl > l) add(--pl);
12    while (pr < r) add(++pr); while (pr > r) del(pr--);
13    ans[qid] = cur;
14 }

```

4.2 CDQ

aaab6c

```

1 // sorted by x; type 0 = update (weight w), 1 = query
2 // query: ans[id] = sum of w over updates with x' <= x,
3 // y' <= y, z' <= z (< if strict[dim]). 值域 [1, mxc]
4 // Dependency: 值域 BIT
5 struct Info { int x, y, z, type; ll w; int id; };
6 struct CDQ {
7     static bool cy(const Info& p, const Info& q)
8     { return p.y < q.y; }
9     vector<Info> a; int n; array<bool, 3> strict;
10    vector<ll> ans; BIT bit;
11    void dfs(int l, int r) { if (r - l <= 1) return;
12      int m = (l + r) / 2, i = l;
13      dfs(l, m);
14      vector<Info> t(a.begin() + m, a.begin() + r);
15      ranges::sort(t, cy);
16      for (auto& q : t) {
17          for (; i < m && a[i].y < q.y + !strict[1]; i++)
18              if (!a[i].type) bit.upd(a[i].z, a[i].w);
19          if (q.type)
20              ans[q.id] += bit.qry(q.z - strict[2]);
21      }
22      for (int k = l; k < i; k++)
23          if (!a[k].type) bit.upd(a[k].z, -a[k].w);
24      dfs(m, r);
25      inplace_merge(a.begin() + l, a.begin() + m,
26        a.begin() + r, cy);
27 }
28 CDQ(vector<Info> _a, int mxc, array<bool, 3> _s = {}):
29     a(_a), n(a.size()), strict(_s) {
30     ans.assign(n, 0); bit.init(mxc);
31     auto key = [&](Info& p) { return
32       make_tuple(p.x, p.type != strict[0], p.y, p.z); };
33     ranges::sort(a, {}, key); dfs(0, n);
34 }
35 };

```

4.3 Persistent Treap

f03e62

```

1 // Author: Ian
2 struct node {
3     node *l, *r;
4     char c; int v, sz;
5     node(char x = '$'): c(x), v(mt()), sz(1) {
6         l = r = nullptr;
7     }
8     node(node* p) { *this = *p; }
9     void pull() {
10         sz = 1;
11         for (auto i : {l, r})
12             if (i) sz += i->sz;
13     }
14 } arr[maxn], *ptr = arr;
15 inline int size(node* p) { return p ? p->sz : 0; }
16 node* merge(node* a, node* b) {
17     if (!a || !b) return a ? a : b;
18     if (a->v < b->v) {
19         node* ret = new(ptr++) node(a);
20         ret->r = merge(ret->r, b); ret->pull();
21         return ret;
22     }
23     else {
24         node* ret = new(ptr++) node(b);
25         ret->l = merge(a, ret->l); ret->pull();
26         return ret;
27     }
28 }
29 P<node*> split(node* p, int k) {
30     if (!p) return {nullptr, nullptr};
31     if (k >= size(p->l) + 1) {
32         auto [a, b] = split(p->r, k - size(p->l) - 1);
33         node* ret = new(ptr++) node(p);
34         ret->r = a, ret->pull();
35         return {ret, b};
36     }
37     else {
38         auto [a, b] = split(p->l, k);
39         node* ret = new(ptr++) node(p);
40         ret->l = b, ret->pull();
41         return {a, ret};
42     }
43 }

```

4.4 Li Chao Tree

bd358d

```

1 // Author: Ian
2 // insert(L): add line L(ax+b) in [0, maxn)
3 // insertseg(L, ql, qr): add segment L in [ql, qr)
4 // query(x): find maximum L at x
5 using T=long long;
6 const int maxn = 1e5+5;
7 const T MN = -1e15;
8 struct line {
9     T a, b;
10    line(T a=0, T b=MN): a(a), b(b) {}
11    T operator()(T x) { return a * x + b; }
12 } arr[(maxn+1)<<3];
13 bool operator<(line a, line b) {
14     return a.a < b.a || (a.a == b.a && a.b < b.b);
15 }
16 #define m ((l+r)>>1)
17 void insert(line x, int i=1, int l=0, int r=maxn) {
18     if (r-l == 1) {
19         if (x(l) > arr[i](l)) arr[i] = x;
20         return;
21     }
22     line a = max(arr[i], x), b = min(arr[i], x);
23     if (a(m) > b(m))
24         arr[i] = a, insert(b, i<<1, l, m);
25     else
26         arr[i] = b, insert(a, i<<1|1, m, r);
27 }
28 void insertseg(line x, int ql, int qr,
29                 int i=1, int l=0, int r=maxn) {
30     if (qr<=l || r<=ql || ql>=qr) return;
31     if (ql<=l && r<=qr) { insert(x, i, l, r); return; }
32     insertseg(x, ql, qr, i<<1, l, m);
33     insertseg(x, ql, qr, i<<1|1, m, r);
34 }
35 T query(T x, int i=1, int l=0, int r=maxn) {
36     if (x<l || r<=x) return MN;
37     if (r-l == 1) return arr[i](x);
38     return max({arr[i](x),
39                 query(x, i<<1, l, m), query(x, i<<1|1, m, r)});
40 }
41 #undef m

```

4.5 Time Segment Tree

c6cb66

```

1 // Author: Ian
2 struct Op; // reversible operation

```

```

3 const int maxn = 2e5 + 5;
4 vector<Op> arr[(maxn + 1) << 2];
5 #define m ((l + r) >> 1)
6 void insert(int ql, int qr, Op x, int i=1, int l=0, int r=maxn) {
7     if (qr <= l || r <= ql) return;
8     if (ql <= l && r <= qr) { arr[i].push_back(x); return; }
9     if (qr <= m) insert(ql, qr, x, i << 1, l, m);
10    else if (m <= ql) insert(ql, qr, x, i << 1 | 1, m, r);
11    else {
12        insert(ql, qr, x, i << 1, l, m);
13        insert(ql, qr, x, i << 1 | 1, m, r);
14    }
15 }
16 void traversal(vector<int>& ans, int i=1, int l=0, int r=maxn) {
17     int opcnt = 0;
18     for (auto val : arr[i])
19         if (DO_SOMETHING(val)) opcnt++;
20     if (r - l == 1) ans[l] = CALC_ANS();
21     else {
22         traversal(ans, i << 1, l, m);
23         traversal(ans, i << 1 | 1, m, r);
24     }
25     while (opcnt--) UNDO_SOMETHING();
26     arr[i].clear();
27 }
28 #undef m

```

4.6 Segment Tree Beats

9938f4

```

1 // Author: Ian, https://judge.yosupo.jp/submission/400861
2 const int maxn=2e5+5;
3 const ll MX = 1e18+5;
4 const ll MN = -MX;
5 #define m ((l+r)>>1)
6 struct node {
7     ll sum, inc;
8     ll mn, smn; int mnc;
9     ll mx, smx; int mxc;
10    node(ll x=0): sum(x), inc(0),
11           mn(x), smn(MX), mnc(1),
12           mx(x), smx(MN), mxc(1) {}
13    void pull(int i);
14    void push(int i, int l, int r);
15 } a[(maxn+1)<<2];
16 void node::pull(int i) {
17     sum = a[i<<1].sum+a[i<<1|1].sum;
18     if (a[i<<1].mn == a[i<<1|1].mn) {
19         mn = a[i<<1].mn;
20         smn = min(a[i<<1].smn, a[i<<1|1].smn);
21         mnc = a[i<<1].mnc+a[i<<1|1].mnc;
22     } else {
23         if (a[i<<1].mn < a[i<<1|1].mn) {
24             mn = a[i<<1].mn;
25             smn = min(a[i<<1].smn, a[i<<1|1].smn);
26             mnc = a[i<<1].mnc;
27         } else {
28             mn = a[i<<1|1].mn;
29             smn = min(a[i<<1|1].smn, a[i<<1].smn);
30             mnc = a[i<<1|1].mnc;
31         }
32     }
33     if (a[i<<1].mx == a[i<<1|1].mx) {
34         mx = a[i<<1].mx;
35         smx = max(a[i<<1].smx, a[i<<1|1].smx);
36         mxc = a[i<<1].mxc+a[i<<1|1].mxc;
37     } else {
38         if (a[i<<1].mx > a[i<<1|1].mx) {
39             mx = a[i<<1].mx;
40             smx = max(a[i<<1].smx, a[i<<1|1].smx);
41             mxc = a[i<<1].mxc;
42         } else {
43             mx = a[i<<1|1].mx;
44             smx = max(a[i<<1|1].smx, a[i<<1].smx);
45             mxc = a[i<<1|1].mxc;
46         }
47     }
48 }
49 void updmn(int i, int len, ll v) {
50     a[i].sum += (v-a[i].mn)*a[i].mnc; a[i].mn = v;
51     if (len==1 || v>=a[i].mx)
52         a[i].mx = v, a[i].smx = MN;
53     else if (v>a[i].smx)
54         a[i].smx = v;
55 }
56 void updmx(int i, int len, ll v) {
57     a[i].sum -= (a[i].mx-v)*a[i].mxc; a[i].mx = v;
58     if (len==1 || v<=a[i].mn)
59         a[i].mn = v, a[i].smn = MX;
60     else if (v<a[i].smn)
61         a[i].smn = v;
62 }
63 void node::push(int i, int l, int r) {
64     if (r-l==1) return;

```

```

63 pair<int, int> ch[] = {{i<<1, m-l}, {i<<1|1, r-m}};
64 if (inc!=0) for (auto [j, len]: ch) {
65     a[j].sum += inc*len, a[j].inc += inc;
66     a[j].mx += inc, a[j].smx += inc;
67     a[j].mn += inc, a[j].smn += inc;
68 } inc = 0;
69 for (auto [j, len]: ch) {
70     if (a[j].mn < mn) updmn(j, len, mn);
71     if (a[j].mx > mx) updmx(j, len, mx); }
72 }
73 void build(vector<ll>& v, int i=1, int l=0, int r=maxn) {
74     if ((int)v.size()<=l) return;
75     if (r-l==1) { a[i] = node(v[l]); return; }
76     build(v, i<<1, l, m);
77     build(v, i<<1|1, m, r);
78     a[i].pull(i);
79 }
80 void chmin(int ql, int qr, ll v, int i=1, int l=0, int r=maxn){
81     if (qr<=l || r<=ql || a[i].mx<=v) return;
82     if (ql<=l && r<=qr && a[i].smx<v) {updmx(i, r-l, v);return;}
83     a[i].push(i, l, r);
84     chmin(ql, qr, v, i<<1, l, m);
85     chmin(ql, qr, v, i<<1|1, m, r);
86     a[i].pull(i);
87 }
88 void chmax(int ql, int qr, ll v, int i=1, int l=0, int r=maxn){
89     if (qr<=l || r<=ql || a[i].mn>=v) return;
90     if (ql<=l && r<=qr && a[i].smn>v) {updmn(i, r-l, v);return;}
91     a[i].push(i, l, r);
92     chmax(ql, qr, v, i<<1, l, m);
93     chmax(ql, qr, v, i<<1|1, m, r);
94     a[i].pull(i);
95 }
96 void inc(int ql, int qr, ll v, int i=1, int l=0, int r=maxn){
97     if (qr<=l || r<=ql) return;
98     if (ql<=l && r<=qr) {
99         a[i].sum += v*(r-l);
100         a[i].mx += v, a[i].smx += v;
101         a[i].mn += v, a[i].smn += v;
102         a[i].inc += v;
103         return;
104     }
105     a[i].push(i, l, r);
106     inc(ql, qr, v, i<<1, l, m);
107     inc(ql, qr, v, i<<1|1, m, r);
108     a[i].pull(i);
109 }
110 ll qsum(int ql, int qr, int i=1, int l=0, int r=maxn) {
111     if (qr<=l || r<=ql) return 0;
112     if (ql<=l && r<=qr) return a[i].sum;
113     a[i].push(i, l, r);
114     return qsum(ql, qr, i<<1, l, m)+qsum(ql, qr, i<<1|1, m, r);
115 }
116 #undef m

```

5 DP

5.1 SOS DP

d14872

```

1// Author: Gino
2// Function: Solve problems that enumerates subsets of
subsets (3^n => n*2^n)
3 for (int msk = 0; msk < (1<<n); msk++) {
4     for (int i = 1; i <= n; i++) {
5         if (msk & (1<<(i - 1))) {
6             dp[msk][i] = dp[msk][i - 1] + dp[msk ^ (1<<(i - 1))][i - 1];
7         } else {
8             dp[msk][i] = dp[msk][i - 1];
9         } }

```

5.2 Divide and Conquer DP

d7f9a9

```

1/* Author: Simon Lindholm (License: CC0)
2 * Description: Given $a[i] = \min_{lo(i) \le k < hi(i)} (f(i, k))$ where the (minimal)
3 * optimal $k$ increases with $i$, computes $a[i]$ for $i = L..R$.
4 * Time: $O((N + (hi-lo)) \log N)$ */
5 struct DP { // Modify at will:
6     int lo(int ind) { return 0; }
7     int hi(int ind) { return ind; }
8     ll f(int ind, int k) { return dp[ind][k]; }
9     void store(int ind, int k, ll v) { res[ind] = pii(k, v); }
10    void rec(int L, int R, int LO, int HI) {
11        if (L >= R) return;
12        int mid = (L + R) >> 1;
13        pair<ll, int> best(LLONG_MAX, LO);
14        rep(k, max(LO, lo(mid)), min(HI, hi(mid)))
15            best = min(best, make_pair(f(mid, k), k));
16        store(mid, best.second, best.first);
17        rec(L, mid, LO, best.second+1);
18        rec(mid+1, R, best.second, HI);

```

```

19 }
20 void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
21 };

```

5.3 Dynamic DP

- 適用情境：\$dp_i = M_i \cdot dp_{i-1} \Rightarrow dp_i = M_i M_{i-1} \dots M_1 dp_0\$
- 當 \$M_i\$ 需要動態修改，且 \$M_i\$ 是廣義矩陣乘法：

$$C_{ij} = \bigoplus_k A_{ik} \otimes B_{kj}$$

- 滿足 \$(\oplus, \otimes)\$ 是半環 (\$\otimes\$ 對 \$\oplus\$ 有分配律、\$\otimes\$ 和 \$\oplus\$ 有結合律、\$\oplus\$ 有交換律)
- 此時便可用線段樹維護 \$M_n M_{n-1} \dots M_1\$ 的乘積。線段樹上的 pos 是 \$M_{pos}\$，但 pull 時要 \$st[idx] = mul(st[cr], st[cl])\$

5.4 Convex Hull Trick

182553

```

1/**
2 * Author: Simon Lindholm (License: CC0, Source: own work)
3 * Container where you can add lines of the form $kx+m$,
4 * and query maximum values at points $x$.
5 * Useful for dynamic programming ('convex hull trick').
6 * Time: $O(\log N)$, Status: stress-tested */
7 struct Line {
8     mutable ll k, m, p;
9     bool operator<(const Line& o) const { return k < o.k; }
10    bool operator<(ll x) const { return p < x; }
11};
12 struct LineContainer : multiset<Line, less<>> {
13     // (for doubles, use inf = 1/.0, div(a,b) = a/b)
14     static const ll inf = LLONG_MAX;
15     ll div(ll a, ll b) { // floored division
16         return a / b - ((a ^ b) < 0 && a % b); }
17    bool isect(iterator x, iterator y) {
18        if (y == end()) return x->p = inf, 0;
19        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
20        else x->p = div(y->m - x->m, x->k - y->k);
21        return x->p >= y->p;
22    }
23    void add(ll k, ll m) {
24        auto z = insert({k, m, 0}), y = z++, x = y;
25        while (isect(y, z)) z = erase(z);
26        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
27        while ((y = x) != begin() && (--x)->p >= y->p)
28            isect(x, erase(y));
29    }
30    ll query(ll x) {
31        auto l = *lower_bound(x);
32        return l.k * x + l.m;
33    }

```

6 Graph

6.1 Max Clique

5621c4

```

1// max N = 64
2 typedef unsigned long long ll;
3 struct MaxClique{
4     ll nb[ N ], n, ans;
5     void init( ll _n ){
6         n = _n;
7         for( int i = 0 ; i < n ; i ++ ) nb[ i ] = 0LLU;
8     }
9     void add_edge( ll _u , ll _v ){
10        nb[ _u ] |= ( 1LLU << _v );
11        nb[ _v ] |= ( 1LLU << _u );
12    }
13    void B( ll r , ll p , ll x , ll cnt , ll res ){
14        if( cnt + res < ans ) return;
15        if( p == 0LLU && x == 0LLU ){
16            if( cnt > ans ) ans = cnt;
17            return;
18        }
19        ll y = p | x; y &= ~y;
20        ll q = p & ( ~nb[ int( log2( y ) ) ] );
21        while( q ){
22            ll i = int( log2( q & (-q) ) );
23            B( r | ( 1LLU << i ) , p & nb[ i ] , x & nb[ i ]
24                , cnt + 1LLU , __builtin_popcountll( p &
25                    nb[ i ] ) );
26            q &= ~( 1LLU << i );
27            p &= ~( 1LLU << i );
28            x |= ( 1LLU << i );
29        }
30    }
31    int solve(){
32        ans = 0;
33        ll _set = 0;
34        if( n < 64 ) _set = ( 1LLU << n ) - 1;

```

```

34     else{
35         for( ll i = 0 ; i < n ; i ++ ) _set |= ( 1LLU << i );
36     }
37     B( 0LLU , _set , 0LLU , 0LLU , n );
38     return ans;
39 }
40 } maxClique;

```

6.2 Bellman-Ford

878821

```

1// Author: Gino
2// n, m = #(vertices), #(edges), and vertices are 1-based
3// 1. BellmanFord bf(G, n, m);
4// 2-1: bf.calcDis(s);
5// => bf.dis[u] := shortest dis from s to u
6// => bf.dis[u] := LINF (s can't reach u)
7// => bf.dis[u] := -LINF (dis[u] can be arbitrary small)
8// 2-2: bf.findNegCycle();
9// => false (if no neg cycle)
10// => true (has neg cycle & gives an example: bf.neg_cycle)
11// Time: O(VE), Space: O(V + E)
12const ll LINF = 4e18;
13struct BellmanFord {
14    const vector<vector<pair<int, ll>>& G;
15    int n, m; // #(vertices), #(edges)
16    BellmanFord(const auto& G, int n, int m):G(G), n(n), m(m) {}
17    vector<ll> dis; vector<int> nth_relax, pa;
18    void run_bf(const auto& src) {
19        dis.assign(n+1, LINF);
20        nth_relax.assign(n+1, 0); pa.assign(n+1, -1);
21        for (auto& s : src) dis[s] = 0;
22        for (int rlx = 1; rlx <= n; rlx++) {
23            for (int u = 1; u <= n; u++) {
24                if (dis[u] == LINF) continue; // !important
25                for (auto& [v, w] : G[u]) {
26                    if (dis[v] > dis[u] + w) {
27                        dis[v] = dis[u] + w; pa[v] = u;
28                        if (rlx == n) nth_relax[v] = 1;
29                    } } } } // 5 brackets
30    void calcDis(int s) {
31        run_bf(vector<int>{s}); queue<int> q;
32        for (int u=1; u<=n; u++) if (nth_relax[u]) q.push(u);
33        while (!q.empty()) { int u = q.front(); q.pop();
34            for (auto& [v, w] : G[u]) {
35                if (!nth_relax[v]) {
36                    nth_relax[v] = 1; q.push(v);
37                } } }
38        for (int u=1; u<=n; u++) if (nth_relax[u]) dis[u] = -LINF;
39    }
40    vector<int> neg_cycle;
41    bool findNegCycle() {
42        auto src = views::iota(1, n + 1); run_bf(src);
43        auto it = ranges::find_if(src,
44            [&](int s){ return nth_relax[s]; });
45        if (it == src.end()) return false;
46        int ptr = *it; for (int i=0; i<n; i++) ptr = pa[ptr];
47        neg_cycle.clear(); int cur = ptr;
48        while (true) {
49            neg_cycle.emplace_back(cur);
50            if (cur == ptr && neg_cycle.size() > 1) break;
51            cur = pa[cur];
52        }
53        ranges::reverse(neg_cycle); return true;
54    } }

```

6.3 System of Difference Constraints

371c50

```

1vector<vector<pair<int, ll>>& G;
2void add(int u, int v, ll w) { G[u].push_back({v, w}); }

```

- $x_u - x_v \leq c \Rightarrow \text{add}(v, u, c)$
- $x_u - x_v \geq c \Rightarrow \text{add}(u, v, -c)$
- $x_u - x_v = c \Rightarrow \text{add}(v, u, c), \text{add}(u, v, -c)$
- $x_u \geq c \Rightarrow \text{add super vertex } x_0 = 0, \text{ then } x_u - x_0 \geq c \Rightarrow \text{add}(u, 0, -c)$
- Don't for get non-negative constraints for every variable if specified implicitly.
- Interval sum $\Rightarrow$ Use prefix sum to transform into differential constraints. Don't for get $S_{i+1} - S_i \geq 0$ if x_i needs to be non-negative.
- $x_u/x_v \leq c \Rightarrow \log x_u - \log x_v \leq \log c$

6.4 Graph Girth

Run BFS for every node, when encountered non-BFS-tree edge, update answer (min cycle length) with $\text{dis}[u] + \text{dis}[v] + 1$. Time $O(VE)$.

6.5 Euler Trail

e959cd

```

1// Author: kactl (Simon Lindholm), wrapped by Gino
2// Vertices are 1-based, Edge ID are 0-based
3// 1. EulerWalk euler(n, true(directed) / false(undirected));
4// 2. euler.addEdge(u, v);

```

```

5// 3. int result = euler.solve();
6// => -1: nothing ( euler.path, euler.path_edges = {} )
7// => 1: Euler trail,
8// => 2: Euler circuit
9//     euler.path = {u_1, ..., u_{m+1}}
10//     euler.path_edges = {e_1, ..., e_m}
11// Time: O(V + E), Space: O(V + E)
12// Test: V, E <= 2e5, 99ms
13struct EulerWalk {
14    int n, m = 0; bool dir;
15    vector<vector<pii>> G; vi deg;
16    EulerWalk(int n, bool dir) :
17        n(n), dir(dir), G(n+1), deg(n+1) {}
18    void addEdge(int u, int v) {
19        G[u].push_back({v, m});
20        if (!dir) G[v].push_back({u, m});
21        deg[u]++, deg[v] += dir ? -1 : 1; m++;
22    }
23    vi path, path_edges;
24    void walk(int src) {
25        path.clear(); path_edges.clear();
26        vi its(n+1), visE(m);
27        function<void(int, int)> dfs = [&](int u, int e_in) {
28            while (its[u] < sz(G[u])) {
29                auto [v, e] = G[u][its[u]++];
30                if (visE[e]) continue;
31                visE[e] = 1;
32                dfs(v, e);
33            }
34            path.push_back(u);
35            if (e_in != -1) path_edges.push_back(e_in);
36        };
37        dfs(src, -1);
38        if (sz(path) != m+1) {
39            path.clear(); path_edges.clear(); return;
40        }
41        ranges::reverse(path); ranges::reverse(path_edges);
42    }
43    int solve() {
44        int s = -1, odd = 0, src = 0, dst = 0;
45        for (int u = 1; u <= n; u++) {
46            if (!dir && (deg[u] & 1)) odd++, s = u;
47            if (dir && deg[u] == 1) s = u, src++;
48            if (dir && deg[u] == -1) dst++;
49            if (dir && abs(deg[u]) > 1) return -1;
50        }
51        if (!dir && odd != 0 && odd != 2) return -1;
52        if (dir && !((src == 0 && dst == 0) || (src == 1 && dst == 1))) return -1;
53        if (s == -1) for (int u = 1; u <= n; u++)
54            if (!G[u].empty()) { s = u; break; }
55        if (s == -1) s = 1;
56        walk(s);
57        return path.empty() ? -1 :
58            path.front() == path.back() ? 2 : 1;
59    } }
60 }

```

6.6 Vertex BCC (Round Square Tree)

207b54

```

1// Author: Ian, https://judge.yosupo.jp/submission/399308
2// Vertex BCC (Round Square Tree) 0-based; Edge ID 0-based
3// 1. BCC bcc(n); // n = #(vertices)
4// 2. bcc.add_edge(u, v);
5// 3. bcc.buiId();
6// => bcc[id]: vertices in bcc `id` (square node n+id in T)
7// => T[id]: tree adjacency; square nodes are n..n+ncnt-1
8// => is_cut(x): whether x is a cut vertex
9// => bcc_id(u, v): square node containing tree-edge (u, v)
10struct BCC {
11    int n, m, t, ncnt;
12    vector<vector<pii>> G;
13    vector<vector<int>> T, bcc;
14    vector<int> dep, low, pa2, dep2, s;
15    void add_edge(int u, int v) {
16        G[u].eb(v, m), G[v].eb(u, m++); }
17    void dfs(int x, int f) {
18        if (~dep[x]) return;
19        dep[x] = low[x] = t++;
20        s.push_back(x);
21        for (auto [y, e]: G[x]) if (e != f) {
22            if (dep[y] == -1) {
23                dfs(y, e), low[x] = min(low[x], low[y]);
24                if (low[y] >= dep[x]) {
25                    int id = n + ncnt++; p;
26                    bcc.resize(bcc.size()+1);
27                    do {
28                        p = s.back(), s.pop_back();
29                        T[id].push_back(p), T[p].push_back(id);
30                        bcc.back().push_back(p);
31                    } while (p != y);

```

```

32     T[id].push_back(x), T[x].push_back(id);
33     bcc.back().push_back(x);
34 }
35 } else low[x] = min(low[x], dep[y]); } }
36 void dfs2(int x, int p) {
37     dep2[x] = ~p ? dep2[p] + 1 : 0, pa2[x] = p;
38     for (auto y: T[x]) if (y != p) dfs2(y, x); }
39 bool is_cut(int x) { return (int)T[x].size() != 1; }
40 int bcc_id(int u, int v) {
41     return pa2[dep2[u] < dep2[v] ? v : u] - n; }
42 void build() {
43     for (int i = 0; i < n; i++) if (dep[i] == -1) {
44         if (G[i].empty()) { // isolated vertex: singleton bcc
45             dep[i] = low[i] = t++;
46             int id = n + ncnt++;
47             bcc.push_back({i});
48             T[id].push_back(i), T[i].push_back(id);
49         } else dfs(i, -1);
50         dfs2(i, -1);
51     } }
52 BCC(int n): n(n), m(0), t(0), ncnt(0), G(n), T(2*n),
53     dep(n, -1), low(n), pa2(2*n), dep2(2*n) {}
54 };

```

6.7 Edge BCC

de44ee

```

1 // Author: Ian, https://judge.yosupo.jp/submission/399295
2 // Edge BCC, 0-based; Edge ID 0-based
3 // 1. EBCC ebcc(n); // n = #(vertices)
4 // 2. ebcc.add_edge(u, v);
5 // 3. ebcc.build();
6 // => bcc[i]: each edge bcc
7 // => bccn[x]: in which bcc is x
8 // => isbr[e]: whether edge e is a bridge
9 struct EBCC {
10     int n, m, t, ncnt;
11     vector<vector<pii>> G;
12     vector<vector<int>> bcc;
13     vector<bool> isbr;
14     vector<int> dep, low, bccn, s;
15     EBCC(int n): n(n), m(0), t(0), ncnt(0), G(n),
16         dep(n, -1), low(n), bccn(n) {}
17     void add_edge(int u, int v) {
18         G[u].eb(v, m), G[v].eb(u, m++); }
19     void dfs(int x, int f) {
20         if (~dep[x]) return;
21         dep[x] = low[x] = t++;
22         s.push_back(x);
23         for (auto [y, e]: G[x]) if (e != f) {
24             if (dep[y] == -1)
25                 dfs(y, e), low[x] = min(low[x], low[y]);
26             else
27                 low[x] = min(low[x], dep[y]);
28         }
29         if (dep[x] == low[x]) {
30             if (~f) isbr[f] = true;
31             bcc.resize(bcc.size()+1);
32             int p; do {
33                 bccn[p = s.back()] = ncnt;
34                 s.pop_back();
35                 bcc.back().push_back(p);
36             } while (p != x); ncnt++;
37         } }
38     void build() {
39         isbr.assign(m, false);
40         for (int i=0; i<n; i++) dfs(i, -1); }
41 };

```

6.8 Kth Shortest Path

b340af

```

1 // time: O(|E| |lg|E|+|V| |lg|V|+K)
2 // memory: O(|E| |lg|E|+|V|)
3 struct KSP { // 1-base
4     struct nd {
5         int u, v; ll d;
6         nd(int ui=0, int vi=0, ll di=INF) { u=ui; v=vi; d=di; }
7     };
8     struct heap { nd* edge; int dep; heap* chd[4]; };
9     static int cmp(heap* a, heap* b)
10     { return a->edge->d > b->edge->d; }
11     struct node {
12         int v; ll d; heap* H; nd* E;
13         node() {}
14         node(ll _d, int _v, nd* _E) { d=_d; v=_v; E=_E; }
15         node(heap* _H, ll _d) { H=_H; d=_d; }
16         friend bool operator<(node a, node b)
17         { return a.d > b.d; }
18     };
19     int n, k, s, t, dst[N]; nd *nxt[N];
20     vector<nd*> g[N], rg[N]; heap *nullNd, *head[N];
21     void init(int _n, int _k, int _s, int _t) {
22         n=_n; k=_k; s=_s; t=_t;

```

```

23         for (int i=1; i<=n; i++) {
24             g[i].clear(); rg[i].clear();
25             nxt[i]=NULL; head[i]=NULL; dst[i]=-1;
26         }
27         void addEdge(int ui, int vi, ll di) {
28             nd* e=new nd(ui, vi, di);
29             g[ui].push_back(e); rg[vi].push_back(e);
30         }
31         queue<int> dfsQ;
32         void dijkstra() {
33             while (dfsQ.size()) dfsQ.pop();
34             priority_queue<node> Q; Q.push(node(0, t, NULL));
35             while (!Q.empty()) {
36                 node p=Q.top(); Q.pop(); if (dst[p.v] != -1) continue;
37                 dst[p.v]=p.d; nxt[p.v]=p.E; dfsQ.push(p.v);
38                 for (auto e: rg[p.v]) Q.push(node(p.d+e->d, e->u, e));
39             }
40             heap* merge(heap* curNd, heap* newNd) {
41                 if (curNd==nullNd) return newNd;
42                 heap* root=new heap; memcpy(root, curNd, sizeof(heap));
43                 if (newNd->edge->d < curNd->edge->d) {
44                     root->edge=newNd->edge;
45                     root->chd[2]=newNd->chd[2];
46                     root->chd[3]=newNd->chd[3];
47                     newNd->edge=curNd->edge;
48                     newNd->chd[2]=curNd->chd[2];
49                     newNd->chd[3]=curNd->chd[3];
50                 }
51                 if (root->chd[0]->dep < root->chd[1]->dep)
52                     root->chd[0]=merge(root->chd[0], newNd);
53                 else root->chd[1]=merge(root->chd[1], newNd);
54                 root->dep=max(root->chd[0]->dep,
55                     root->chd[1]->dep)+1;
56                 return root;
57             }
58         vector<heap*> V;
59         void build() {
60             nullNd=new heap; nullNd->dep=0; nullNd->edge=new nd;
61             fill(nullNd->chd, nullNd->chd+4, nullNd);
62             while (not dfsQ.empty()) {
63                 int u=dfsQ.front(); dfsQ.pop();
64                 if (!nxt[u]) head[u]=nullNd;
65                 else head[u]=head[nxt[u]->v];
66                 V.clear();
67                 for (auto& e: g[u]) {
68                     int v=e->v;
69                     if (dst[v] == -1) continue;
70                     e->d+=dst[v]-dst[u];
71                     if (nxt[u] != e) {
72                         heap* p=new heap; fill(p->chd, p->chd+4, nullNd);
73                         p->dep=1; p->edge=e; V.push_back(p);
74                     }
75                 }
76                 if (V.empty()) continue;
77                 make_heap(V.begin(), V.end(), cmp);
78                 #define L(X) ((X<<1)+1)
79                 #define R(X) ((X<<1)+2)
80                 for (size_t i=0; i<V.size(); i++) {
81                     if (L(i)<V.size()) V[i]->chd[2]=V[L(i)];
82                     else V[i]->chd[2]=nullNd;
83                     if (R(i)<V.size()) V[i]->chd[3]=V[R(i)];
84                     else V[i]->chd[3]=nullNd;
85                 }
86                 head[u]=merge(head[u], V.front());
87             } }
88         vector<ll> ans;
89         void first_K() {
90             ans.clear(); priority_queue<node> Q;
91             if (dst[s] == -1) return;
92             ans.push_back(dst[s]);
93             if (head[s] != nullNd)
94                 Q.push(node(head[s], dst[s]+head[s]->edge->d));
95             for (int _=1; _<k and not Q.empty(); _++) {
96                 node p=Q.top(); Q.pop(); ans.push_back(p.d);
97                 if (head[p.H->edge->v] != nullNd) {
98                     q.H=head[p.H->edge->v]; q.d=p.d+q.H->edge->d;
99                     Q.push(q);
100                 }
101             }
102             for (int i=0; i<4; i++)
103                 if (p.H->chd[i] != nullNd) {
104                     q.H=p.H->chd[i];
105                     q.d=p.d+p.H->edge->d+p.H->chd[i]->edge->d;
106                     Q.push(q);
107                 } }
108         void solve() { // ans[i] stores the i-th shortest path
109             dijkstra(); build();
110             first_K(); // ans.size() might less than k

```

111} solver;

6.9 SCC - Tarjan

298974

```

1 // Author: Ian, https://judge.yosupo.jp/submission/399291
2 // Tarjan SCC, 0-based
3 // 1. SCC scc(n); // n = #(vertices)
4 // 2. Fill in adj list G
5 // 3. scc.build()
6 // => scc[i]: each scc
7 // => sccn[x]: in which scc is x
8 struct SCC {
9     const int inf = 1e9+5;
10    int n, t, ncnt;
11    vector<vector<int>> G, scc;
12    vector<bool> ins;
13    vector<int> sccn, dfn, low, s;
14    SCC(int n): n(n), t(0), ncnt(0), G(n),
15        ins(n, false), sccn(n, -1), low(n, inf) {}
16    void dfs(int x) {
17        if (~dfn[x]) return;
18        dfn[x] = low[x] = t++;
19        s.push_back(x), ins[x] = true;
20        for (auto i: G[x])
21            if (dfs(i), ins[i])
22                low[x] = min(low[x], low[i]);
23        if (dfn[x] == low[x]) {
24            scc.resize(scc.size()+1);
25            int p; do {
26                ins[p = s.back()] = false;
27                s.pop_back(); sccn[p] = ncnt;
28                scc.back().push_back(p);
29            } while (p != x); ncnt++;
30        }
31    }
32    void build() { for (int i=0; i<n; i++) dfs(i); }
33};

```

6.10 2SAT

e5fb57

```

1 // Author: Ian, CRYPTOGrapheR
2 struct TwoSAT: SCC {
3     TwoSAT(int n): SCC(2*n) {}
4     // (var a == na) v (var b == nb)
5     void add_disjunction(int a, int na, int b, int nb) {
6         a = 2*a^na, b = 2*b^nb;
7         G[a^1].push_back(b);
8         G[b^1].push_back(a);
9     }
10    vector<int> solve() {
11        build(); vector<int> assignment(n/2, 0);
12        for (int i=0; i<n/2; i++) {
13            if (sccn[2*i] == sccn[2*i^1]) return {};
14            assignment[i] = sccn[2*i] > sccn[2*i^1];
15        } return assignment;
16    };

```

7 Tree**7.1 Tree Isomorphism (Rooted Trees)**

83f86d

```

1 // Author: Gino
2 // Checks if 2 rooted trees are isomorphic
3 // Time: O(V log V)
4 // vector<vector<int>> G1, G2;
5 // int root_1, root_2;
6 map<vector<int>, int> dict;
7 int encode(const auto& G, int u, int pa = -1) {
8     vector<int> codes;
9     for (auto& v: G[u])
10         if (v != pa)
11             codes.emplace_back(encode(G, v, u));
12     ranges::sort(codes);
13     auto [it, _] = dict.try_emplace(codes, dict.size());
14     return it->second;
15 }
16 bool isomorphic() {
17     dict.clear(); dict[{}] = 0;
18     return encode(G1, root_1) == encode(G2, root_2);
19 }

```

7.2 Tree Isomorphism (Unrooted Trees)Find the centroid(s) of T_1, T_2 .Case 1: T_1, T_2 have different number of centroids $\rightarrow$ NOCase 2: T_1 has centroid c_1, T_2 has centroid c_2 $\rightarrow$ rooted_isomorphic(c_1, c_2)Case 3: T_1 has centroids c_1, c'_1, T_2 has centroids c_2, c'_2 $\rightarrow$ rooted_isomorphic(c_1, c_2) || rooted_isomorphic(c'_1, c'_2)**7.3 Heavy Light Decomposition**

6518aa

```

1 // Author: Ian, https://judge.yosupo.jp/submission/399317
2 void segbuild(vector<int>&v);
3 void segmodify(int p, int k);
4 int segquery(int ql, int qr);

```

5 // Insert [ql, qr) segment tree here

```

6 template<class T=int> struct HLD {
7     int n, cnt;
8     vector<int> d, f, sz, son, top, dfn;
9     HLD(int n): n(n), cnt(0), d(n, 0), f(n, 0), sz(n, 1),
10         son(n, -1), top(n, 0), dfn(n, -1) {}
11     void dfs1(int x, int p) {
12         for (auto i: e[x]) if (i != p) {
13             d[i] = d[x]+1, f[i] = x;
14             dfs1(i, x), sz[x] += sz[i];
15             if (son[x] == -1 || sz[son[x]] < sz[i])
16                 son[x] = i;
17         }
18     }
19     void dfs2(int x, int t) {
20         dfn[x] = cnt++, top[x] = t;
21         if (son[x] == -1) return;
22         dfs2(son[x], t);
23         for (auto i: e[x])
24             if (!dfn[i]) dfs2(i, i);
25     }
26     void build(const vector<T> &v) {
27         dfs1(0, 0); dfs2(0, 0);
28         vector<T> dfnv(v.size());
29         for (int i=0; i<n; i++)
30             dfnv[dfn[i]] = v[i];
31         segbuild(dfnv);
32     }
33     void modify(int p, T v) { segmodify(dfn[p], v); }
34     T query(int a, int b) {
35         auto aggr = [](int x, int y) { return max(x, y); };
36         T ret = 0;
37         while (top[a] != top[b]) {
38             if (d[top[a]] > d[top[b]]) swap(a, b);
39             ret = aggr(ret, segquery(dfn[top[b]], dfn[b]+1));
40             b = f[top[b]];
41         }
42         if (dfn[a] > dfn[b]) swap(a, b);
43         return aggr(ret, segquery(dfn[a], dfn[b]+1));
44     }
45 };

```

7.4 Virtual Tree

cbbdb82

```

1 // Author: std_abs
2 // need lca, in, out
3 vector<pii> virtual_tree(vector<int> &v) {
4     auto cmp = [&](int x, int y) { return in[x] < in[y]; };
5     sort(all(v), cmp);
6     for (int i = 1; i < sz(v); ++i)
7         v.pb(lca(v[i-1], v[i]));
8     sort(all(v), cmp);
9     v.resize(unique(all(v)) - v.begin());
10    vector<int> stk(1, v[0]);
11    vector<pii> res;
12    for (int i = 1; i < sz(v); ++i) {
13        int x = v[i];
14        while (out[stk.back()] < out[x]) stk.pop_back();
15        res.emplace_back(stk.back(), x), stk.pb(x);
16    }
17    return res;
18 }

```

8 Matching**8.1 Bipartite Matching**

0e5f0d

```

1 // Author: Gino
2 // Function: Max Bipartite Matching in O(V sqrt(E))
3 //> hk.init(nx, ny) -> hk.add(x, y (+nx))
4 //> hk.max_matching(): the matching plan stores in mx, my
5 //> hk.min_vertex_cover(): vertex cover plan stores in vcover
6 // (!) vertices are 0-based: X = [0, nx), Y = [nx, nx+ny)
7 #define pb emplace_back
8 struct HopcroftKarp {
9     int n, nx, ny; vector<vector<int>> G; vector<int> mx, my;
10    HopcroftKarp(int nx, int ny): nx(nx), ny(ny), n(nx + ny) {}
11    { G.assign(n, vector<int>()); }
12    void add(int x, int y) { G[x].pb(y); G[y].pb(x); }
13    int max_matching() {
14        vector<int> dis, vis;
15        mx.assign(n, -1); my.assign(n, -1);
16        function<bool(int)> dfs = [&](int x) {
17            vis[x] = 1;
18            for (int y: G[x]) { int p = my[y];
19                if (p == -1 ||
20                    (dis[p] == dis[x] + 1 && !vis[p] && dfs(p)))
21                    return mx[x] = y, my[y] = x, true;
22            }
23            return false;
24        };
25        while (true) {
26            dis.assign(n, -1);
27            queue<int> q;

```

```

28 for (int x = 0; x < nx; x++)
29     if (mx[x] == -1) dis[x] = 0, q.push(x);
30 while (!q.empty()) {
31     int x = q.front(); q.pop();
32     for (int y : G[x])
33         if (my[y] != -1 && dis[my[y]] == -1)
34             dis[my[y]] = dis[x] + 1, q.push(my[y]);
35 }
36 vis.assign(n, 0);
37 bool found = false;
38 for (int x = 0; x < nx; x++)
39     if (mx[x] == -1 && dfs(x)) found = true;
40 if (!found) break;
41 }
42 int ans = 0;
43 for (int x = 0; x < nx; x++) if (mx[x] != -1) ans++;
44 return ans;
45 }
46 vector<int> vcover;
47 int min_vertex_cover() {
48     int ans = max_matching(); vector<int> vis(n, 0);
49     function<void(int)> dfs = [&](int x) {
50         vis[x] = true;
51         for (int y : G[x]) {
52             if (y == mx[x] || my[y] == -1 || vis[y]) continue;
53             vis[y] = true;
54             dfs(my[y]);
55         }
56         for (int x = 0; x < nx; x++) if (mx[x] == -1) dfs(x);
57         vcover.clear();
58         for (int x = 0; x < nx; x++) if (!vis[x]) vcover.pb(x);
59         for (int y = nx; y < nx + ny; y++) if (vis[y]) vcover.pb(y);
60         return ans;
61     };

```

8.2 Bipartite Weighted Matching

2b5b69

```

1// Author: ckiseki, n = #(left vertices) = #(right vertices)
2// left, right vertices labeled 1 ~ n separately
3// 1. build 1-based matrix G of size (n + 1) * (n + 1)
4// G[u][v] := weight between (left u, right v)
5// <!-- if not need perfect matching
6// ==> set weights of all negative edges to 0
7// 2. construct (solve at same time): KM km(n, G);
8// ==> km.ans (max perfect weight sum)
9// ==> km.fl[u] (left u -> right v)
10// ==> km.fr[v] (right v -> left u)
11// Time: O(V^3), Space: O(V^2), Test: V <= 500, 243ms
12 struct KM { // maximize, test @ UOJ 80
13     int n, l, r; ll ans; // fl and fr are the match
14     vector<ll> hl, hr; vector<int> fl, fr, pre, q;
15     void bfs(const auto &w, int s) {
16         vector<int> vl(n + 1), vr(n + 1);
17         vector<ll> slk(n + 1, LINF);
18         l = r = 0; vr[q[r++] = s] = true;
19         auto check = [&](int x) -> bool {
20             if (vl[x] || slk[x] > 0) return true;
21             vl[x] = true; slk[x] = LINF;
22             if (fl[x] != -1) return (vr[q[r++] = fl[x]] = true);
23             while (x != -1) swap(x, fr[fl[x] = pre[x]]);
24             return false;
25         };
26         while (true) {
27             while (l < r)
28                 for (int x = 1, y = q[l++]; x <= n; ++x) if (!vl[x])
29                     if ((ll val = hl[x] + hr[y] - w[x][y]; val < slk[x]) {
30                         slk[x] = val;
31                         if (pre[x] = y, !check(x)) return;
32                     }
33             ll d = LINF;
34             for (int x = 1; x <= n; ++x) d = min(d, slk[x]);
35             for (int x = 1; x <= n; ++x)
36                 vl[x] ? hl[x] += d : slk[x] -= d;
37             for (int x = 1; x <= n; ++x) if (vr[x]) hr[x] -= d;
38             for (int x = 1; x <= n; ++x) if (!check(x)) return;
39         }
40         KM(int n_, const auto &w) : n(n_), ans(0),
41             hl(n + 1), hr(n + 1), fl(n + 1, -1), fr(fl), pre(n + 1), q(n + 1) {
42             for (int i = 1; i <= n; ++i) {
43                 hl[i] = -LINF;
44                 for (int j = 1; j <= n; ++j) hl[i] = max(hl[i], (ll)w[i][j]);
45             }
46             for (int i = 1; i <= n; ++i) bfs(w, i);
47             for (int i = 1; i <= n; ++i) ans += w[i][fl[i]];
48         }
49     };

```

8.3 General Matching

2e4962

```

1// Author: ckiseki
2// 0. build 1-based graph vector<vector<int>> G(n + 1);

```

```

3// 1. construct (solve at same time):
4// GeneralMatching M(G, n); ==> M.ans
5// status: M.match[x] (== -1 if not matched, 1 <= x <= n)
6// Time: O(V^3), Space: O(V + E), Test: V <= 500, 11ms
7 struct GeneralMatching {
8     queue<int> q; int ans, n;
9     vector<int> fa, s, v, pre, match;
10    int Find(int u) {
11        return u == fa[u] ? u : fa[u] = Find(fa[u]);
12    }
13    int LCA(int x, int y) {
14        static int tk = 0; tk++; x = Find(x); y = Find(y);
15        for (; swap(x, y)) if (x != 0) {
16            if (v[x] == tk) return x;
17            v[x] = tk;
18            x = Find(pre[match[x]]);
19        }
20    }
21    void Blossom(int x, int y, int l) {
22        for (; Find(x) != l; x = pre[y]) {
23            pre[x] = y, y = match[x];
24            if (s[y] == 1) q.push(y), s[y] = 0;
25            for (int z : {x, y}) if (fa[z] == z) fa[z] = l;
26        }
27    }
28    bool Bfs(auto &&g, int r) {
29        iota(all(fa), 0); ranges::fill(s, -1);
30        q = queue<int>(); q.push(r); s[r] = 0;
31        for (; !q.empty(); q.pop()) {
32            for (int x = q.front(); int u : g[x])
33                if (s[u] == -1) {
34                    if (pre[u] = x, s[u] = 1, match[u] == 0) {
35                        for (int a = u, b = x, last;
36                             b != 0; a = last, b = pre[a])
37                            last = match[b], match[b] = a, match[a] = b;
38                        return true;
39                    }
40                    q.push(match[u]); s[match[u]] = 0;
41                }
42            else if (!s[u] && Find(u) != Find(x)) {
43                int l = LCA(u, x);
44                Blossom(x, u, l); Blossom(u, x, l);
45            }
46        }
47        return false;
48    }
49    GeneralMatching(auto &&g, int n_) : ans(0), n(n_),
50        fa(n + 1), s(n + 1), v(n + 1), pre(n + 1, 0), match(n + 1, 0) {
51        for (int x = 1; x <= n; ++x)
52            if (match[x] == 0) ans += Bfs(g, x);
53        for (int x = 1; x <= n; ++x)
54            if (match[x] == 0) match[x] = -1;
55    };

```

9 Flow

9.1 Flow Methods

9516b6

```

1// Author: CRyptographER (some modified by Gino)
2Maximize  $c^T x$  subject to  $Ax \leq b, x \geq 0$ ;
3with the corresponding symmetric dual problem,
4Minimize  $b^T y$  subject to  $A^T y \geq c, y \geq 0$ .
5Maximize  $c^T x$  subject to  $Ax \leq b$ ;
6with the corresponding asymmetric dual problem,
7Minimize  $b^T y$  subject to  $A^T y = c, y \geq 0$ .
8Maximize  $\sum x$  subject to  $x_i + x_j \leq A_{ij}, x \geq 0$ ;
9=> Maximize  $\sum x$  subject to  $x_i + x_j \leq A_{ij}$ ;
10=> Minimize  $A^T y = \sum A_{ij} y_{ij}$  subject to for all  $v$ ,
11 $\sum_{i=v \text{ or } j=v} y_{ij} = 1, y_{ij} \geq 0$ 
12=> possible optimal solution:  $y_{ij} = \{0, 0.5, 1\}$ 
13=>  $y=2y: \sum_{i=v \text{ or } j=v} y'_{ij} = 2, y'_{ij} = \{0, 1, 2\}$ 
14=> Minimum Bipartite perfect matching/2 ( $V_1=X, V_2=X, E=A$ )
15General Graph:
16|Max Ind. Set| + |Min Vertex Cover| = |V|
17|Max Ind. Edge Set| + |Min Edge Cover| = |V|
18Bipartite Graph:
19|Max Ind. Set| = |Min Edge Cover|
20|Max Ind. Edge Set| = |Min Vertex Cover|
21To reconstruct the minimum vertex cover, dfs from each
22unmatched vertex on the left side and with unused edges
23only. Equivalently, dfs from source with unused edges
24only and without visiting sink. Then, a vertex is
25chosen iff. it is on the left side and without visited
26or on the right side and visited through dfs.
27Minimum Weighted Bipartite Edge Cover:
28Construct new bipartite graph with  $n+m$  vertices on each
29side:
30for each vertex  $u$ , duplicate a vertex  $u'$  on the other side
31for each edge  $(u, v, w)$ , add edges  $(u, v, w)$  and  $(v', u', w)$ 
32for each vertex  $u$ , add edge  $(u, u', 2w)$  where  $w$  is min edge
33connects to  $u$ 
34then the answer is the minimum perfect matching of the new
35graph (KM)
36Maximum density subgraph ( $\sum W_e + \sum W_v$ ) / |V|
37Binary search on answer:

```

34 For a fixed D, construct a Max flow model as follow:
 35 Let S be Sum of all weight(or inf)
 36 1. from source to each node with cap = S
 37 2. For each (u,v,w) in E, (u->v, cap=w), (v->u, cap=w)
 38 3. For each node v, from v to sink with cap = S + 2 * D - deg[v] - 2 * (W of v)
 39 where deg[v] = \sum weight of edge associated with v
 40 If maxflow < S * |V|, D is an answer.
 41 Requiring subgraph: all vertex can be reached from source with
 42 edge whose cap > 0.
 43 Maximum closed subgraph
 44 1. connect source with positive weighted vertex(capacity=weight)
 45 2. connect sink with negative weighted vertex(capacity=-weight)
 46 3. make capacity of the original edges = inf
 47 4. ans = sum(positive weighted vertex weight) - (max flow)
 48 (Node-disjoint) Min DAG Path Cover (用最短路覆蓋所有點)
 49 Node disjoint: 拆出來的路徑不能共用同一個點
 50 將一個點 u 裂成 u+ 和 u- , 代表進入和出去
 51 對於一條邊 (u, v) 建邊 u- => v+
 52 1. +點們和 -點們形成一張二分圖
 53 2. 最差的答案是 n , 代表每個點自己一個點就是一條路徑
 54 3. 只要一組 (u-, v+) 匹配成功那對應到的答案恰好會 -1
 55 >>> ans = n - 這張二分圖的最大匹配
 56 General DAG Path Cover
 57 跟 Node-disjoint 版本差在點可以被很多條路共用
 58 建邊方式 Node-disjoint 差別在條件比較鬆
 59 Node-disjoint: u- => v+ 只能在 (u, v) 有邊時建
 60 General: u- => v+ 在 u 能走到 v 的時候就建
 61 Dilworth Theorem
 62 反鏈: 一些節點的集合, 滿足這些節點互相無法抵達
 63 最大反鏈 = 最小 General DAG Path Cover
 64 Min Cut reconstruction (after running max flow s -> t):
 65 From s, BFS/DFS through residual edges with remaining
 66 capacity > 0, marking reached vertices as visited.
 67 The cut set is every original edge (u, v) with u visited
 68 and v not visited; its capacity sum equals the max flow.
 69 Bipartite Matching reconstruction (after running max flow
 70 on source -> X -> Y -> sink model):
 71 For each edge (x, y) between the X-side and Y-side, it is
 72 part of the matching iff. its residual capacity is 0
 73 (i.e. it is saturated).
 74 Max/Min Flow with Lower Bound (Circulation Problem):
 75 1. Construct super source S and super sink T.
 76 2. For each edge (x, y, l, u), connect x -> y with capacity
 77 u - l.
 78 3. For each vertex v, let in(v) = (sum of incoming lower
 79 bounds) - (sum of outgoing lower bounds).
 80 4. If in(v) > 0, connect S -> v with capacity in(v);
 81 otherwise connect v -> T with capacity -in(v).
 82 To maximize: connect t -> s with capacity inf (skip this
 83 edge
 84 for a pure circulation problem), let f = maxflow(S, T).
 85 If
 86 f != sum of in(v) over all in(v) > 0, no solution exists.
 87 Otherwise maxflow(s, t) (found after adding t -> s) is
 88 the answer.
 89 To minimize: let f = maxflow(S, T) (without any s-t edge
 90 yet).
 91 Then connect t -> s with capacity inf and let f' be the
 92 extra
 93 flow pushed from S to T. If f + f' != sum of in(v) over
 94 all
 95 in(v) > 0, no solution exists. Otherwise f' is the
 96 answer.
 97 5. The flow on each original edge e is l_e + f_e, where f_e
 98 is
 99 e's flow in the constructed graph.

9.2 Dinic (Flow Decomposition)

55581b

```
1// Function: Max Flow + Flow Decomposition, O(V^2 E)
2// init -> addEdge; vertices 0-based
3// >>> flow() := return max flow
4// >>> flow_decomposition() := return max flow + store
  decomposition in flow.D
5struct Dinic{
6  static const int MXN=10000;
7  struct Edge{ int v,c,r,C; bool fw; };
8  int n,s,t,level[MXN]; vector<Edge> G[MXN];
9  void init(int _n,int _s,int _t){
10     n=_n; s=_s; t=_t;
11     for (int i=0; i<n; i++) G[i].clear(); }
12  void addEdge(int u,int v,int c){
13     G[u].push_back({v,c,(int)G[v].size(),c,true});
14     G[v].push_back({u,0,(int)G[u].size()-1,0,false}); }
15  bool BFS(){
16     for (int i=0; i<n; i++) level[i]=-1;
17     queue<int> que; que.push(s); level[s]=0;
```

```
18     while(!que.empty()){
19         int u=que.front(); que.pop();
20         for (auto &it:G[u]){
21             if(it.c>0&&level[it.v]==-1){
22                 level[it.v]=level[u]+1; que.push(it.v);
23             } }
24         return level[t]!=-1; }
25  int DFS(int u,int nf){
26     if(u==t) return nf;
27     int res=0;
28     for (auto &it:G[u]){
29         if(it.c>0&&level[it.v]==level[u]+1){
30             int tf=DFS(it.v,min(nf,it.c));
31             res+=tf; nf-=tf; it.c-=tf;
32             G[it.v][it.r].c+=tf;
33             if(nf==0) return res;
34         } }
35     if(!res) level[u]=-1;
36     return res; }
37  int flow(int res=0){
38     while(BFS()) res+=DFS(s,2147483647);
39     return res;
40 } // reset: do nothing
41 vector<pair<int, vector<int>>> D; // (flow amount, [path
  p1 ... pk])
42  int flow_decomposition(){
43     int mxflow = flow();
44     vector<vector<Edge>> fG(n); // graph with forward edges
45     for (int u = 0; u < n; u++) {
46         for (auto &e : G[u]) {
47             if (e.fw && e.C - e.c > 0)
48                 fG[u].push_back({e.v, e.C - e.c, e.r, e.C, e.fw});
49         } }
50     vector<int> vis(n, 0);
51     function<int(int, int)> dfs = [&](int u, int cur) {
52         if (u == t) {
53             D.back().second.push_back(u);
54             return cur; }
55         vis[u] = 1;
56         for (auto &e : fG[u]) {
57             if (e.c > 0 && !vis[e.v]) {
58                 int ans = dfs(e.v, min(cur, e.c));
59                 if (ans > 0) {
60                     e.c -= ans;
61                     D.back().second.push_back(u);
62                     return ans;
63                 } } }
64         return 0; };
65     D.clear();
66     int quota = mxflow;
67     while (quota > 0) {
68         D.emplace_back(0, vector<int>());
69         fill(vis.begin(), vis.end(), 0);
70         int f = dfs(s, 2147483647);
71         if (f == 0) break;
72         reverse(D.back().second.begin(),
73                D.back().second.end());
74         D.back().first = f, quota -= f;
75     }
76     return mxflow; }
77 }flow;
```

9.3 ISAP

e622e3

```
1// Author: CRYPTOGRAPHER
2// 1-based: vertices are numbered 1 ~ n
3// 1. flow.init(n, s, t);
4// n = #vertices, s, t = source, sink
5// 2. flow.addEdge(u, v, c); // u, v in [1, n]
6// 3. call: int ans = flow.flow();
7// => ans = max flow value from s to t
8// Time: O(V^2 * E) worst case, usually much faster in
  practice
9static const int MAXV=50010;
10static const int INF =1000000;
11struct Maxflow{
12  struct Edge{
13     int v,c,r;
14     Edge(int _v,int _c,int _r):v(_v),c(_c),r(_r){}
15 };
16  int s,t; vector<Edge> G[MAXV];
17  int iter[MAXV],d[MAXV],gap[MAXV],tot;
18  void init(int n,int _s,int _t){
19     tot=n,s=_s,t=_t;
20     for(int i=1;i<=tot;i++){
21         G[i].clear(); iter[i]=d[i]=gap[i]=0;
22     }
23 }
24  void addEdge(int u,int v,int c){
25     G[u].push_back(Edge(v,c,SZ(G[v])));
```

```

26 G[v].push_back(Edge(u,0,SZ[G[u]]-1));
27
28 int DFS(int p,int flow){
29     if(p==t) return flow;
30     for(int &i=iter[p];i<SZ[G[p]];i++){
31         Edge &e=G[p][i];
32         if(e.c>0&&d[p]==d[e.v]+1){
33             int f=DFS(e.v,min(flow,e.c));
34             if(f){ e.c-=f; G[e.v][e.r].c+=f; return f; }
35         }
36     }
37     if(--gap[d[p]]==0) d[s]=tot;
38     else{ d[p]++; iter[p]=0; ++gap[d[p]]; }
39     return 0;
40 }
41 int flow(){
42     int res=0;
43     for(res=0;gap[0]==tot;d[s]<tot;res+=DFS(s,INF));
44     return res;
45 }
46 void reset(){
47     for(int i=1;i<=tot;i++) iter[i]=d[i]=gap[i]=0;
48 }
49 } flow;

```

9.4 Bounded Max Flow

10dd73

```

1// Author: CRYPTOGRAPHER
2// Max flow with lower/upper bound on edges (source-sink
  version)
3// <!-- 1-based: original vertices are numbered 1 ~ n.
4// Super source, sink: n+1, n+2 respectively.
5// <!-- dependency: ISAP.cpp
6// 1. n = #vertices (1 ~ n), m = #edges
7// s, t = original source, sink (both in [1, n])
8// 2. l[i], r[i] := edge i goes from l[i] -> r[i] (l[i],
  r[i] in [1, n])
9// 3. a[i], b[i] := flow on edge i must lie in [a[i], b[i]]
10// 4. int ans = solve(n, m, s, t);
11// => ans == -1 : no feasible flow exists
12// => ans : feasible max flow value
13// <!-- N, M must be large enough for original graph + 2
  super nodes
14// Time: O(V^2 E), Space: O(V + E)
15int in[N], out[N], l[M], r[M], a[M], b[M];
16int solve(int n, int m, int s, int t){
17    flow.init(n+2, n+1, n+2); // super source = n+1, super
  sink = n+2
18    for(int i = 0; i < m; i++){
19        in[r[i]] += a[i]; out[l[i]] += a[i];
20        flow.addEdge(l[i], r[i], b[i] - a[i]);
21        // flow from l[i] to r[i] must lie in [a[i], b[i]]
22    }
23    int nd = 0;
24    for(int i = 1; i <= n; i++){
25        if(in[i] < out[i]){
26            flow.addEdge(i, flow.t, out[i] - in[i]);
27            nd += out[i] - in[i];
28        }
29        if(out[i] < in[i])
30            flow.addEdge(flow.s, i, in[i] - out[i]);
31    }
32    flow.addEdge(t, s, INF); // original sink -> source
33    if(flow.flow() != nd) return -1; // infeasible
34    int ans = flow.G[s].back().c; // t->s edge's current flow
  = feasible flow s->t
35    flow.G[s].back().c = flow.G[t].back().c = 0;
36    for(size_t i = 0; i < flow.G[flow.s].size(); i++){
37        Maxflow::Edge &e = flow.G[flow.s][i];
38        flow.G[flow.s][i].c = 0; flow.G[e.v][e.r].c = 0;
39    }
40    for(size_t i = 0; i < flow.G[flow.t].size(); i++){
41        Maxflow::Edge &e = flow.G[flow.t][i];
42        flow.G[flow.t][i].c = 0; flow.G[e.v][e.r].c = 0;
43    }
44    flow.addEdge(flow.s, s, INF); flow.addEdge(t, flow.t,
  INF);
45    flow.reset(); // keeps G[] intact, only resets search
  state
46    return ans + flow.flow();
47 }

```

9.5 MCMF

9f4297

```

1// Author: CRYPTOGRAPHER
2// MCMF (supports negative edges without negative cycles)
3// <!-- 1-based: vertices are numbered 1 ~ n.
4// 1. MCMF.init(n); // n = #vertices (1 ~ n)
5// 2. MCMF.add(u, v, cap, cost);
6// 3. auto [max_flow, min_cost] = MCMF.flow(s, t); // s, t
  in [1, n]
7// Time: O(F * E), Space: O(V + E)
8typedef int Tcost;

```

```

9const int MAXV = 20010;
10const int INFf = 1000000;
11const Tcost INFc = 1e9;
12struct MCMF {
13    struct Edge{
14        int v, cap;
15        Tcost w;
16        int rev;
17        bool fw;
18        Edge(){}
19        Edge(int t2, int t3, Tcost t4, int t5, bool t6)
20            : v(t2), cap(t3), w(t4), rev(t5), fw(t6) {}
21    };
22    int V, s, t;
23    vector<Edge> G[MAXV];
24    void init(int n){
25        V = n;
26        for(int i = 0; i <= V; i++) G[i].clear();
27    }
28    void add(int a, int b, int cap, Tcost w){
29        G[a].push_back(Edge(b, cap, w, (int)G[b].size(), true));
30        G[b].push_back(Edge(a, 0, -w, (int)G[a].size()-1,
  false));
31    }
32    Tcost d[MAXV];
33    int id[MAXV], mom[MAXV];
34    bool inqu[MAXV];
35    queue<int> q;
36    pair<int, Tcost> flow(int _s, int _t){
37        s = _s, t = _t;
38        int mxf = 0, Tcost mnc = 0;
39        while(1){
40            fill(d, d+1+V, INFc); // need to use type cast
41            fill(inqu, inqu+1+V, 0);
42            fill(mom, mom+1+V, -1);
43            mom[s] = s;
44            d[s] = 0;
45            q.push(s); inqu[s] = 1;
46            while(q.size()){
47                int u = q.front(); q.pop();
48                inqu[u] = 0;
49                for(int i = 0; i < (int) G[u].size(); i++){
50                    Edge &e = G[u][i];
51                    int v = e.v;
52                    if(e.cap > 0 && d[v] > d[u]+e.w){
53                        d[v] = d[u]+e.w;
54                        mom[v] = u;
55                        id[v] = i;
56                        if(!inqu[v]) q.push(v), inqu[v] = 1;
57                    }
58                }
59            }
60            if(mom[t] == -1) break;
61            int df = INFf;
62            for(int u = t; u != s; u = mom[u])
63                df = min(df, G[mom[u]][id[u]].cap);
64            for(int u = t; u != s; u = mom[u]){
65                Edge &e = G[mom[u]][id[u]];
66                e.cap -= df;
67                G[e.v][e.rev].cap += df;
68            }
69            mxf += df;
70            mnc += df*d[t];
71        }
72        return make_pair(mxf, mnc);
73    }
74 };

```

9.6 Push-Relabel

f37045

```

1// Author: Simon Lindholm (kactl)
2// 0-based: vertices are numbered 0 ~ n-1
3// 1. PushRelabel pr(n); // n = #vertices
4// 2. pr.addEdge(u, v, c); // u, v in [0, n-1]
5// 3. ll ans = pr.calc(s, t);
6// => ans = max flow value from s to t
7// To obtain actual flow:
8// for (int u = 0; u < n; u++)
9//     for (auto& e : pr.g[u])
10//         if (e.f > 0) => (u, e.dest, e.f)
11// Time: O(V^2 * sqrt(E)), better on dense graph
12struct PushRelabel {
13    struct Edge { int dest, back; ll f, c; };
14    vector<vector<Edge>> g;
15    vector<ll> ec;
16    vector<Edge*> cur;
17    vector<vi> hs; vi H;
18    PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}
19    void addEdge(int s, int t, ll cap, ll rcap=0) {
20        if (s == t) return;

```

```

21 g[s].push_back({t, sz(g[t]), 0, cap});
22 g[t].push_back({s, sz(g[s])-1, 0, rcap});
23 }
24 void addFlow(Edge& e, ll f) {
25     Edge &back = g[e.dest][e.back];
26     if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
27     e.f += f; e.c -= f; ec[e.dest] += f;
28     back.f -= f; back.c += f; ec[back.dest] -= f;
29 }
30 ll calc(int s, int t) {
31     int v = sz(g); H[s] = v; ec[t] = 1;
32     vi co(2*v); co[0] = v-1;
33     rep(i, 0, v) cur[i] = g[i].data();
34     for (Edge& e : g[s]) addFlow(e, e.c);
35     for (int hi = 0; ) {
36         while (hs[hi].empty()) if (!hi--) return -ec[s];
37         int u = hs[hi].back(); hs[hi].pop_back();
38         while (ec[u] > 0) // discharge u
39             if (cur[u] == g[u].data() + sz(g[u])) {
40                 H[u] = 1e9;
41                 for (Edge& e : g[u]) if (e.c && H[u] >
42                     H[e.dest]+1)
43                     H[u] = H[e.dest]+1, cur[u] = &e;
44                 if (++co[H[u]], !--co[hi] && hi < v)
45                     rep(i, 0, v) if (hi < H[i] && H[i] < v)
46                         --co[H[i]], H[i] = v + 1;
47                 hi = H[u];
48             } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
49                 addFlow(*cur[u], min(ec[u], cur[u]->c));
50             else ++cur[u];
51     }
52     bool leftOfMinCut(int a) { return H[a] >= sz(g); }
53 }

```

9.7 Gomory-Hu Tree

7f6c17

```

1 // Author: chilli, Takanori MAEHARA (kactl)
2 // <!-- Dependency: PushRelabel.cpp
3 // Given a list of edges representing an undirected flow
4 // graph,
5 // returns edges of the Gomory-Hu tree.
6 // maxflow / mincut of (u, v) => min edge from u to v on
7 // Gomory-Hu tree
8 // Time: O(V) * O(PushRelabel)
9 typedef array<ll, 3> Edge; // (u, v, w)
10 vector<Edge> gomoryHu(int N, vector<Edge> ed) {
11     vector<Edge> tree;
12     vi par(N);
13     rep(i, 1, N) {
14         PushRelabel D(N); // Dinic also works
15         for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
16         tree.push_back({i, par[i], D.calc(i, par[i])});
17         rep(j, i+1, N)
18             if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
19     }
20     return tree;
21 }

```

9.8 Global Min Cut

f8153b

```

1 // Author: ckiseki
2 // 1-based: vertices are numbered 1 ~ n
3 // 1. vector<vector<ll>> G(n+1, vector<ll>(n+1, 0));
4 // 2. add_edge(G, u, v, c); // add undirected edge u-v with
5 //    weight c, u,v in [1,n]
6 // 3. ll ans = mincut(G, n); // n = #vertices
7 // => ans = global min cut value
8 // Time: O(V^3)
9 void add_edge(auto &w, int u, int v, int c) {
10     w[u][v] += c; w[v][u] += c; }
11 auto phase(const auto &w, int n, vector<int> id) {
12     vector<ll> g(n+1); int s = -1, t = -1;
13     while (!id.empty()) {
14         int c = -1;
15         for (int i : id) if (c == -1 || g[i] > g[c]) c = i;
16         s = t; t = c;
17         id.erase(ranges::find(id, c));
18         for (int i : id) g[i] += w[c][i];
19     }
20     return tuple{s, t, g[t]};
21 }
22 ll mincut(auto w, int n) {
23     ll cut = numeric_limits<ll>::max();
24     vector<int> id(n); iota(all(id), 1);
25     for (int i = 0; i < n - 1; ++i) {
26         auto [s, t, gt] = phase(w, n, id);
27         id.erase(ranges::find(id, t));
28         cut = min(cut, gt);
29         for (int j = 1; j <= n; ++j)
30             w[s][j] += w[t][j], w[j][s] += w[j][t];
31     }
32     return cut;
33 }

```

10 String

10.1 Rolling Hash

adc100

```

1 // Author: ckiseki (Modified by Gino)
2 // Usage: RH<Sigma, Mod> Hs(s);
3 // If no modifications needed, replace BIT with prefix sum
4 template <ll P = 127, ll Q = 1051762951>
5 class RH {
6     int n; vector<ll> p, inv, d, bit;
7     ll add(ll a, ll b) { return a+b >= Q ? a+b-Q : a+b; }
8     ll sub(ll a, ll b) { return a-b < 0 ? a-b+Q : a-b; }
9     ll mul(ll a, ll b) { return (__int128)a * b % Q; }
10    ll pw(ll a, ll x) {
11        ll r = 1; for (; x; a = mul(a, a), x >>= 1)
12            if (x & 1) r = mul(r, a);
13        return r;
14    }
15    void bit_add(int i, ll x)
16        { for (; i <= n; i += i&1) bit[i] = add(bit[i], x); }
17    ll bit_qry(int i)
18        { ll s = 0; for (; i; i -= i&1) s = add(s, bit[i]); }
19    public:
20        RH(const auto &s) : n(s.size()),
21            p(n+1, 1), inv(n+1, 1), d(n+1), bit(n+1) {
22            ll iv = pw(P, Q - 2);
23            for (int i = 1; i <= n; i++) {
24                p[i] = mul(p[i-1], P); inv[i] = mul(inv[i-1], iv);
25                d[i] = mul(s[i-1], p[i]); bit_add(i, d[i]);
26            }
27            void update(int i, char c)
28                { ll x = mul(c, p[i]); bit_add(i, sub(x, d[i])); d[i] = x; }
29            ll query(int l, int r)
30                { return mul(sub(bit_qry(r), bit_qry(l-1)), inv[l]); }
31        };

```

10.2 KMP

7d5bf5

```

1 // Author: Gino
2 vector<int> kmp(auto s) { int n=s.size(); vector<int> f(n,0);
3     for (int i = 1, ptr = 0; i < n; i++) {
4         while (ptr && s[i] != s[ptr]) ptr = f[ptr-1];
5         if (s[i] == s[ptr]) ptr++; f[i] = ptr;
6     }
7     return f;
8 }

```

10.3 Z Value

b28f20

```

1 // Author: std_abs
2 vector<int> Z(auto s) { int n=s.size(); vector<int> Z(n);
3     for (int i = 0, l = 0, r = 0; i < n; ++i) {
4         Z[i] = max(min(Z[i-l], r-i), 0);
5         while (i+Z[i] < n && s[Z[i]] == s[i+Z[i]])
6             l = i, r = i+Z[i], Z[i]++;
7     }
8     return Z;
9 }

```

10.4 Manacher

d1ee05

```

1 // Author: std_abs
2 // return array Z (length == 2N - 1)
3 // Example: s = "aaaa"
4 //    a      a      a      a
5 //    Z_0 Z_1 Z_2 Z_3 Z_4 Z_5 Z_6
6 //    1  2  3  4  3  2  1
7 // The corresponding palindrome s[l..r]:
8 // l = (i - Z[i] + 1) / 2
9 // r = (i + Z[i] - 1) / 2
10 vector<int> manacher(auto tmp) {
11     vector<int> s(1, -1);
12     for (auto c : tmp) s.push_back(c), s.push_back(-1);
13     int l = 0, r = 0, n = (int)s.size();
14     vector<int> Z(n);
15     for (int i = 0; i < n; ++i) {
16         Z[i] = r > i ? min(Z[2 * l - i], r - i) : 1;
17         while (i - Z[i] >= 0 && i + Z[i] < n &&
18             s[i + Z[i]] == s[i - Z[i]]) Z[i]++;
19         if (Z[i] + i > r) l = i, r = Z[i] + i;
20     }
21     for (int i = 0; i < n; ++i)
22         Z[i] = (Z[i] - (i & 1)) / 2 * 2 + (i & 1);
23     return vector<int>(Z.begin() + 1, Z.end() - 1);
24 }

```

10.5 Suffix Array + LCP

a4c835

```

1 // Author: std_abs
2 // Tested @yosupo, N <= 5e5, 123ms
3 const int N = 5e5 + 10;
4 int sa[N], tmp[2][N], c[N], rk[N], lcp[N];
5 void buildSA(string s) {
6     int *x = tmp[0], *y = tmp[1], m = 256, n = s.size();
7     for (int i = 0; i < m; ++i) c[i] = 0;
8     for (int i = 0; i < n; ++i) c[x[i]] = s[i]++;
9     for (int i = 1; i < m; ++i) c[i] += c[i-1];

```

```

10 for (int i = n - 1; ~i; --i) sa[--c[x[i]]] = i;
11 for (int k = 1; k < n; k <= 1) {
12     for (int i = 0; i < m; ++i) c[i] = 0;
13     for (int i = 0; i < n; ++i) c[x[i]]++;
14     for (int i = 1; i < m; ++i) c[i] += c[i - 1];
15     int p = 0;
16     for (int i = n - k; i < n; ++i) y[p++] = i;
17     for (int i = 0; i < n; ++i) if (sa[i] >= k)
18         y[p++] = sa[i] - k;
19     for (int i = n - 1; ~i; --i)
20         sa[--c[x[y[i]]]] = y[i];
21     y[sa[0]] = p = 0;
22     for (int i = 1; i < n; ++i) {
23         int a = sa[i], b = sa[i - 1];
24         p += !(x[a] == x[b] && a + k < n && b + k < n && x[a + k] == x[b + k]);
25         y[sa[i]] = p;
26     }
27     if (n == p + 1) break;
28     swap(x, y), m = p + 1;
29 } }
30 void buildLCP(string s) {
31     // lcp[i] = LCP(sa[i - 1], sa[i])
32     // lcp(i, j) = query_lcp_min [rk[i] + 1, rk[j] + 1)
33     int n = s.length(), x = 0;
34     for (int i = 0; i < n; ++i) rk[sa[i]] = i;
35     for (int i = 0; i < n; ++i) {
36         if (!rk[i]) lcp[rk[i]] = 0;
37         else { if (x) --x;
38               int p = sa[rk[i] - 1];
39               while (x + i < n && x + p < n && s[x + i] == s[x + p]) x++;
40               lcp[rk[i]] = x;
41         } }

```

10.6 Suffix Automaton

57a8b0

```

1 // Author: std_abs
2 // state (i.e. node) -> all substr with the same endpos
3 // those substrs' length in range [len(link) + 1, len]
4 // state i's endpos set -> union of pos[i] over the subtree
  of i in the suffix link tree
5 // link[i] -> longest suffix with different endpos set
  (suffix link tree edge)
6 // len[i] -> length longest suffix of state i
7 // pos[i] -> an example end position of state i (1-based)
8 // cnt[i] -> size of endpos set of state i, built by
  build_cnt() (tree DP)
9 const int N = 5e5 + 10, C = 26;
10 struct SAM {
11     int ch[2*N][C], len[2*N], link[2*N], pos[2*N], cnt[2*N], _id;
12     SAM () { reset(); }
13     void reset() { _id = 0, newnode(), link[0] = -1; }
14     int newnode() {
15         fill_n(ch[_id], C, 0);
16         len[_id] = link[_id] = pos[_id] = cnt[_id] = 0;
17         return _id++;
18     }
19     void build(string s) {
20         int last = 0; for (int i = 0; i < (int)s.size(); i++) {
21             char c = s[i]; int cur = newnode();
22             len[cur] = len[last] + 1, pos[cur] = i + 1;
23             int p = last;
24             while (~p && !ch[p][c - 'a'])
25                 ch[p][c - 'a'] = cur, p = link[p];
26             if (p == -1) link[cur] = 0;
27             else { // conflict: p --> q, p --> cur
28                 int q = ch[p][c - 'a'];
29                 if (len[p] + 1 == len[q]) { // q is suffix of cur
30                     link[cur] = q;
31                 } else { // we need auxiliary state
32                     int nxt = newnode();
33                     len[nxt] = len[p] + 1, link[nxt] = link[q];
34                     pos[nxt] = 0;
35                     for (int j = 0; j < C; j++)
36                         ch[nxt][j] = ch[q][j];
37                     while (~p && ch[p][c - 'a'] == q)
38                         ch[p][c - 'a'] = nxt, p = link[p];
39                     link[q] = link[cur] = nxt;
40                 } }
41             cnt[cur]++, last = cur;
42         } }
43 } sam;
44 // void build_cnt() {
45 //     vector<int> p(_id), iota(all(p), 0); sort(all(p),
46 //         [&](int i, int j){ return len[i] > len[j]; });
47 //     for (int i = 0; i < _id; ++i) if (link[p[i]] != -1)
48 //         cnt[link[p[i]]] += cnt[p[i]];
49 // }
50 // bool match(string s) {
51 //     int p = 0; for (auto& c : s) {
52 //         if (!ch[p][c - 'a']) return false;

```

```

53 //     p = ch[p][c - 'a'];
54 // } return true; }

```

10.7 Minimum Rotation

47e675

```

1 // Author: std_abs
2 string rotate(auto s) { int n=s.size(), i=0, j=1;
3     string t = s + s;
4     while (i < n && j < n) {
5         int k = 0; while (k < n && t[i + k] == t[j + k]) ++k;
6         if (t[i + k] <= t[j + k]) j += k + 1; else i += k + 1;
7         if (i == j) ++j;
8     } return t.substr((i < n ? i : j), n);
9 }

```

10.8 AC Automaton

a67b1f

```

1 // Author: std_abs (slightly modified by Gino)
2 // Recap: fail[s] = s 在 trie 上 (也就是某個 pattern 的前綴
  的最長 proper 後綴
3 // N := sum(length of all patterns) + 1
4 // 1. ac.insert(pat) -> end-node id recorded in ac.pid
5 // 2. ac.build_fail()
6 // 3. run text: ptr = to[ptr][c-'a'] each char;
  accumulate on your own arrays
9 // - pull: rep(i ac.order_cnt-1 ~ 0) val[fail[x]] += val[x]
10 // - push: rep(i 0 ~ ac.order_cnt-1) val[x] += val[fail[x]]
11 // Tested @yosupo, N <= 1e6, 323ms on building AC
12 const int N = 1e6 + 5, C = 26;
13 struct AC {
14     int ch[N][C], to[N][C], fail[N], _id, order[N], order_cnt;
15     int np; vector<int> pid; // pattern ID on trie
16     // order[r] := which trie node has BFS order r
17     // fail link tree: (parent) fail[i] <--- i (child)
18     void reset() { _id = np = order_cnt = 0, pid.clear(),
19         newnode(); }
19     int newnode() {
20         fill_n(ch[_id], C, 0), fill_n(to[_id], C, 0);
21         fail[_id] = 0; return _id++;
22     }
23     AC () { reset(); }
24     void insert(string s) {
25         int ptr = 0; for (char c : s) {
26             if (!ch[ptr][c - 'a']) ch[ptr][c - 'a'] = newnode();
27             ptr = ch[ptr][c - 'a'];
28         } np++; pid.emplace_back(ptr);
29     }
30     void build_fail() {
31         queue<int> q; order_cnt = 0;
32         for (int i = 0; i < C; ++i) if (ch[0][i])
33             q.push(ch[0][i]), to[0][i] = ch[0][i];
34         while (!q.empty()) {
35             int u = q.front(); q.pop(); order[order_cnt++] = u;
36             for (int i = 0; i < C; ++i) {
37                 if (!ch[u][i]) to[u][i] = to[fail[u]][i];
38                 else { int v = ch[u][i], k = fail[u];
39                     while (k && !ch[k][i]) k = fail[k];
40                     if (ch[k][i]) k = ch[k][i];
41                     fail[v] = k, to[u][i] = v;
42                     q.push(v);
43                 } } }
44 } ac;

```

10.9 AC Automaton - Dict Links

2e252f

```

1 // Author: Gino, paste this in `struct AC {...}`
2 // 3 Modifications to the original AC Automaton:
3 // int np; vector<int> pid; bool is_pattern[N];
4 // fail[_id] = 0, is_pattern[_id] = false; return _id++;
5 // np++; pid.emplace_back(ptr); is_pattern[ptr] = true;
6 int dict[N];
7 void build_dict() { // call this after build_fail()!
8     for (int i = 0; i < order_cnt; i++) {
9         int u = order[i], f = fail[u];
10         dict[u] = is_pattern[f] ? f : dict[f];
11     }

```

10.10 AC Automaton - Counting Patterns

abf7fe

```

1 // Author: Gino, paste this in `struct AC {...}`
2 ll val[N];
3 void count_patterns(string& text, vector<string>& patterns) {
4     for (auto& p : patterns) insert(p); build_fail();
5     ranges::fill(val, 0LL);
6     int ptr = 0; for (auto& c : text) {
7         ptr = to[ptr][c - 'a']; val[ptr]++;
8     }
9     for (int x : span(order, order_cnt) | views::reverse)
10         val[fail[x]] += val[x];
11     for (auto& i : pid) cout << val[i] << endl;
12 }

```

10.11 AC Automaton - Pattern Position

輸出每個 pattern 在 text 中第一次出現的位置，若不存在則輸出 -1。
走 AC 自動機時 if (val[ptr] == -1) val[ptr] = i; 接著 pull DP

```

if (val[x] == -1) continue;
if (val[fail[x]] == -1) val[fail[x]] = val[x];
else val[fail[x]] = min(val[fail[x]], val[x]);

```

11 Geometry

11.1 Template

600601

```

1 using T = long long; // or double
2 const double eps = 1e-8;
3 int sgn(T x) { return (x > eps) - (x < -eps); }
4 struct Pt {
5     T x = 0, y = 0;
6     Pt operator+(Pt a) const { return {x + a.x, y + a.y}; }
7     Pt operator-(Pt a) const { return {x - a.x, y - a.y}; }
8     Pt operator*(T k) const { return {x * k, y * k}; }
9     Pt operator/(T k) const { return {x / k, y / k}; }
10    T operator*(Pt a) const { return x * a.x + y * a.y; }
11    T operator^(Pt a) const { return x * a.y - y * a.x; }
12    bool operator<(Pt a) const
13    { return tie(x, y) < tie(a.x, a.y); }
14    bool operator==(Pt a) const
15    { return !sgn(x - a.x) && !sgn(y - a.y); }
16    // friend ostream& operator<<(ostream& o, Pt a)
17    // { return o << "(" << a.x << ", " << a.y << ")"; }
18 };
19 struct Line { Pt a, b; }; // a -> b
20 T abs2(Pt a) { return a * a; }
21 double abs(Pt a) { return sqrt(abs2(a)); }
22 int ori(Pt o, Pt a, Pt b) // 1: CCW, -1: CW, 0: collinear
23 { return sgn((a - o) ^ (b - o)); }

```

11.2 Basic

f714a8

```

1 struct Cir { Pt o; double r; };
2 // ----- 極角排序 -----
3 int ud(Pt a) // 0: upper half (incl. +x axis), 1: lower
4 { return !sgn(a.y) ? sgn(a.x) < 0 : a.y < 0; }
5 bool cmp(Pt a, Pt b) // polar angle in [0, 2PI),
6 { return ud(a) == ud(b) ? sgn(a ^ b) > 0 : ud(a) < ud(b); }
7 ranges::sort(p, [&](Pt a, Pt b) { return cmp(a - o, b - o); });
8 // -----
9 bool sameVec(Pt a, Pt b, int d) // d=0: parallel, d=1: same dir
10 { return !sgn(a ^ b) && sgn(a * b) > d * 2 - 2; }
11 bool sameVec(Line a, Line b, int d)
12 { return sameVec(a.b - a.a, b.b - b.a, d); }
13 Pt rot90(Pt a) { return {-a.y, a.x}; } // CCW
14 Pt ref(Pt a) { return ud(a) ? Pt(-a.x, -a.y) : a; }
15 T dbarea(vector<Pt> &p) { // doubled signed area, CCW > 0
16     T res = 0; for (int i = 0; i < sz(p); i++)
17         res += p[i] ^ p[(i+1) % sz(p)];
18     return res; }
19 bool onseg(Pt p, Pt a, Pt b) // p on segment ab?
20 { return !ori(p, a, b) && sgn((a - p) * (b - p)) <= 0; }
21 // ---- T = double ----
22 const double PI = acos(-1);
23 double normalize(double x) { // to [0, 2pi)
24     while (x < -eps) x += PI * 2;
25     while (x > PI * 2 + eps) x -= PI * 2;
26     return x; }
27 double theta(Pt a) { return normalize(atan2(a.y, a.x)); }
28 Pt unit(Pt a) { return a / abs(a); }
29 Pt rot(Pt a, double o) { // CCW
30     double c = cos(o), s = sin(o);
31     return Pt(c * a.x - s * a.y, s * a.x + c * a.y); }
32 Pt projVec(Pt a, Pt b, Pt c) // vector ac proj to ab
33 { return (b - a) * ((c - a) * (b - a) / abs2(b - a)); }
34 Pt projPt(Pt a, Pt b, Pt c) // point c proj to ab
35 { return projVec(a, b, c) + a; }

```

11.3 Line / Segment Intersections

5d5bb0

```

1 // m=0: segment, m=1: ray from l.a to l.b, m=2: line
2 bool banana(Line ls, int ms, Line lt, int mt, int strict) {
3     auto on = [&](Line l, int m, Pt p) {
4         if (ori(l.a, l.b, p) != 0) return false;
5         if (m && abs2(l.a - p) > abs2(l.b - p)) return true;
6         return m == 2 || sgn((p - l.a) * (p - l.b)) <= -strict;
7     };
8     if (sameVec(ls, lt, 0)) {
9         return on(ls, ms, lt.a) || on(ls, ms, lt.b) ||
10            on(lt, mt, ls.a) || on(lt, mt, ls.b);
11     }
12     auto good = [&](Line l, int m, Line o) {
13         if (m && abs((l.a - o.a) ^ (l.a - o.b)) >
14             abs((l.b - o.a) ^ (l.b - o.b))) return true;
15         return m == 2 ||
16            ori(l.a, o.a, o.b) * ori(l.b, o.a, o.b) == -1;
17     };
18     if (good(ls, ms, lt) && good(lt, mt, ls)) return 1;
19     if (!strict) {
20         if (mt != 2 && on(ls, ms, lt.a)) return 1;
21         if (mt == 0 && on(ls, ms, lt.b)) return 1;
22         if (ms != 2 && on(lt, mt, ls.a)) return 1;

```

```

23     if (ms == 0 && on(lt, mt, ls.b)) return 1;
24 }
25 return 0;
26 }
27 // becareful when two lines are parallel
28 auto bananaPoint(Line a, Line b) {
29     auto abc = (a.b - a.a) ^ (b.a - a.a);
30     auto abd = (a.b - a.a) ^ (b.b - a.a);
31     return make_pair(b.b * abc - b.a * abd, abc - abd);
32 }

```

11.4 Point-Line / Segment Distance

fb1271

```

1 // T = double
2 double pointLineDist(Pt p, Line L)
3 { return fabs(((L.b - L.a) ^ (p - L.a)) / abs(L.b - L.a)); }
4 double pointSegDist(Pt p, Pt a, Pt b) {
5     if (a == b) return abs(p - a);
6     if (sgn((b - a) * (p - a)) >= 0 &&
7         sgn((a - b) * (p - b)) >= 0)
8         return pointLineDist(p, {a, b});
9     return min(abs(p - a), abs(p - b));
10 }

```

11.5 Sort

3515b8

```

1 // Dependency: cmp(), btwAngle()
2 // cmp in Basic: polar angle sort
3 // all points are on line ab. closer to a: front
4 bool cmpLine(Pt s, Pt t, Pt a, Pt b) {
5     Pt v = a - b;
6     if (sgn(v.x)) return sgn(s.x - t.x) == sgn(v.x);
7     return sgn(s.y - t.y) == sgn(v.y);
8 }
9 // intersect points polar angle sort, deno: positive
10 bool cmpFractionPolar(pair<Pt, ll> o,
11     pair<Pt, ll> s, pair<Pt, ll> t) {
12     Pt u = s.first * o.second - o.first * s.second;
13     Pt v = t.first * o.second - o.first * t.second;
14     return cmp(u, v);
15 }
16 struct Seg {
17     Pt a, b; // a.x < b.x
18     bool operator<(const Seg& o) const {
19         if (a == o.a) return ori(o.b, a, b) == 1;
20         if (a.x <= o.a.x) return ori(o.a, a, b) == 1;
21         return ori(a, o.a, o.b) == -1;
22     };
23 }
24 struct PolarSeg {
25     Pt a, b; // ori(Pt(0, 0), a, b) > 0
26     bool operator<(const PolarSeg& o) const {
27         if (a == o.a) return ori(o.b, a, b) == -1;
28         if (btwAngle(Pt(0, 0), a, b, o.a, 0))
29             return ori(o.a, a, b) == -1;
30         return ori(a, o.a, o.b) == 1;
31     };
32 }
33 struct Arc {
34     // contain(a, b): circle a inside circle b
35     Cir c; int id, s; // 0 -> up, 1 -> down
36     bool operator<(const Arc& b) const {
37         if (id == b.id) return s < b.s;
38         if (contain(c, b.c)) return b.s == 1;
39         if (contain(b.c, c)) return s == 0;
40         if (c.o.y == b.c.o.y) return id < b.id;
41         return c.o.y > b.c.o.y;
42     };
43 }

```

11.6 Convex Hull

268463

```

1 auto convexHull(vector<Pt> pts) {
2     sort(all(pts)); vector<Pt> ans = {pts[0]};
3     for (int t = 0; t < 2; t++, reverse(all(pts))) {
4         for (int i = 1, m = sz(ans); i < sz(pts); i++) {
5             while (sz(ans) > m &&
6                 ori(ans[sz(ans) - 2], ans.back(), pts[i]) <= 0)
7                 ans.pop_back();
8             ans.pb(pts[i]); } }
9     if (sz(ans) > 1) ans.pop_back(); return ans; }

```

11.7 Triangle Centers

c7e988

```

1 // T = double
2 Pt circenter(Pt a, Pt b, Pt c) { // r = abs(o - a)
3     b = b - a, c = c - a; double m = 2 * (b ^ c);
4     Pt o = {(abs2(b) * c.y - abs2(c) * b.y) / m,
5             (abs2(c) * b.x - abs2(b) * c.x) / m};
6     return o + a;
7 }
8 Pt incenter(Pt a, Pt b, Pt c) { // r = 2 * area / s
9     double x = abs(b - c), y = abs(c - a);
10    double z = abs(a - b), s = x + y + z;
11    return (a * x + b * y + c * z) / s;
12 }
13 Pt centroid(Pt a, Pt b, Pt c) { return (a + b + c) / 3; }

```

```

14 Pt orthocenter(Pt a, Pt b, Pt c) {
15     return centroid(a, b, c) * 3 - circenter(a, b, c) * 2;
16 }

```

11.8 Point Inside

7ff410

```

1 // get edge index: check (0, a), (0, b) first
2 // then after binary search, check (a, b)
3 bool inConvex(vector<Pt>& C, Pt p, bool strict = true) {
4     // only works when no three points are collinear
5     int a = 1, b = sz(C) - 1, r = !strict;
6     if (sz(C) == 0) return false;
7     if (sz(C) < 3) return r && onseg(p, C[0], C.back());
8     if (ori(C[0], C[a], C[b]) > 0) swap(a, b);
9     if (ori(C[0], C[a], p) >= r ||
10         ori(C[0], C[b], p) <= -r) return false;
11     while (abs(a - b) > 1) {
12         int c = (a + b) / 2;
13         (ori(C[0], C[c], p) > 0 ? b : a) = c;
14     }
15     return ori(C[a], C[b], p) < r;
16 }
17 // need onseg(), -1: out, 0: edge, 1: in
18 int inPoly(vector<Pt> poly, Pt o, int strict) {
19     int cnt = 0;
20     for (int i = 0; i < sz(poly); i++) {
21         Pt a = poly[i], b = poly[(i + 1) % sz(poly)];
22         if (onseg(o, a, b)) return !strict;
23         cnt ^= ((o.y < a.y) - (o.y < b.y)) * ori(o, a, b) > 0;
24     }
25     return cnt ? 1 : -1;
26 }
27 // relation of q with circumcircle; T = long long
28 bool inCC(array<Pt, 3> p, Pt q) {
29     __int128 det = 0;
30     for (int i = 0; i < 3; i++)
31         det += __int128(abs2(p[i]) - abs2(q)) *
32             ((p[(i + 1) % 3] - q) ^ (p[(i + 2) % 3] - q));
33     return det > 0; // in: >0, on: =0, out: <0
34 }

```

11.9 Point In Triangle

70dfcb

```

1 // number of p with a < p < b and ori(p, a, b) < 0
2 int under(Pt a, Pt b) {}
3 // number of points with a < p < b and ori(p, a, b) = 0
4 int edge(Pt a, Pt b) {}
5 // check if this number is calculated
6 bool check(Pt p) {}
7 // number of points that strictly inside the triangle
8 int inTri(array<Pt, 3> arr) {
9     sort(all(arr)); auto [a, b, c] = arr; intx = ori(b, a, c);
10    if (x == 0) return 0;
11    if (x == 1) return under(a, b) + under(b, c) -
12        under(a, c) - edge(a, c);
13    return under(a, c) - under(a, b) -
14        under(b, c) - edge(a, b) - edge(b, c) - check(b);
15 }

```

11.10 Vector In Polygon

4e0130

```

1 // ori(a, b, c) >= 0, valid: "strict" angle from a-b to a-c
2 bool btwAngle(Pt a, Pt b, Pt c, Pt p, int strict) {
3     return ori(a, b, p) >= strict &&
4         ori(a, p, c) >= strict;
5 }
6 // whether vector{cur, p} is in order prv, cur, nxt
7 bool inside(Pt prv, Pt cur, Pt nxt, Pt p, int strict) {
8     if (ori(cur, nxt, prv) >= 0)
9         return btwAngle(cur, nxt, prv, p, strict);
10    return !btwAngle(cur, prv, nxt, p, !strict);
11 }

```

11.11 Polygon Cut

d71c4a

```

1 // Dependency: bananaPoint(); T = double
2 // Cut a convex polygon with line (s -> e), keep right side
3 vector<Pt> polyCut(vector<Pt> poly, Pt s, Pt e) {
4     vector<Pt> res; for (int i = 0; i < sz(poly); i++) {
5         Pt cur = poly[i], prv = i ? poly[i - 1] : poly.back();
6         bool side = ori(s, e, cur) < 0;
7         if (side != (ori(s, e, prv) < 0)) {
8             auto [p, d] = bananaPoint({s, e}, {cur, prv});
9             res.pb(p / d);
10        } if (side) res.pb(cur);
11    } return res;
12 }

```

11.12 Half-Plane Intersection

6d7925

```

1 // Dependency: sameVec(), cmp(); T = long long
2 /* Having solution, check size > 2 */
3 /* --^-- Line.a --^-- Line.b --^-- */
4 auto halfPlaneIntersection(vector<Line> arr) {
5     auto areaPair = [&](Line a, Line b) {
6         return make_pair((a.b - a.a) ^ (b.a - a.a),
7             (a.b - a.a) ^ (b.b - a.a));

```

```

8     };
9     auto isin = [&](Line l0, Line l1, Line l2) {
10        // Check inter(l1, l2) strictly in l0
11        auto [a02X, a02Y] = areaPair(l0, l2);
12        auto [a12X, a12Y] = areaPair(l1, l2);
13        if (a12X - a12Y < 0) a12X *= -1, a12Y *= -1;
14        return (__int128)a02Y * a12X -
15            (__int128)a02X * a12Y > 0; // C^4
16    };
17    sort(all(arr), [&](Line a, Line b) {
18        if (sameVec(a, b, 1))
19            return ori(a.a, a.b, b.b) < 0;
20        return cmp(a.b - a.a, b.b - b.a);
21    });
22    deque<Line> dq(1, arr[0]);
23    auto popBack = [&](int t, Line p) {
24        while (sz(dq) >= t &&
25            !isin(p, dq[sz(dq) - 2], dq.back()))
26            dq.pop_back();
27    };
28    auto popFront = [&](int t, Line p) {
29        while (sz(dq) >= t && !isin(p, dq[0], dq[1]))
30            dq.pop_front();
31    };
32    for (auto p : arr) if (!sameVec(dq.back(), p, 1))
33        popBack(2, p), popFront(2, p), dq.pb(p);
34    popBack(3, dq[0]), popFront(3, dq.back());
35    return vector<Line>(all(dq));
36 }

```

11.13 Cyclic Ternary Search

a9f4c0

```

1 /* bool pred(int a, int b);
2 f(0) ~ f(n - 1) is a cyclic-shift U-function
3 return idx s.t. pred(x, idx) is false forall x*/
4 int cycTsearch(int n, auto pred) {
5     if (n == 1) return 0;
6     int l = 0, r = n; bool rv = pred(1, 0);
7     while (r - l > 1) {
8         int m = (l + r) / 2;
9         if (pred(0, m) ? rv : pred(m, (m + 1) % n)) r = m;
10        else l = m;
11    } return pred(l, r % n) ? l : r % n;
12 }

```

11.14 Circle Intersections

226689

```

1 // Dependency: cycTsearch(); T = double for circles
2 vector<Pt> circleLineIntersect(Cir c, Line l) {
3     Pt p = l.a + (l.b - l.a) *
4         ((c.o - l.a) * (l.b - l.a)) / abs2(l.b - l.a);
5     double s = (l.b - l.a) ^ (c.o - l.a);
6     double h2 = c.r * c.r - s * s / abs2(l.b - l.a);
7     if (sgn(h2) == -1) return {};
8     if (sgn(h2) == 0) return {p};
9     Pt h = (l.b - l.a) / abs(l.b - l.a) * sqrt(h2);
10    return {p - h, p + h};
11 }
12 // covered area of c1: arc from res[0] to res[1], CCW
13 vector<Pt> circlesIntersect(Cir c1, Cir c2) {
14     double d2 = abs2(c1.o - c2.o), d = sqrt(d2);
15     if (d < max(c1.r, c2.r) - min(c1.r, c2.r) ||
16         d > c1.r + c2.r) return {};
17     Pt u = (c1.o + c2.o) / 2 +
18         (c1.o - c2.o) *
19         ((c2.r * c2.r - c1.r * c1.r) / (2 * d2));
20     double A = sqrt((c1.r + c2.r + d) *
21         (c1.r - c2.r + d) * (c1.r + c2.r - d) *
22         (-c1.r + c2.r + d));
23     Pt v = rot90(c2.o - c1.o) * A / (2 * d2);
24     if (sgn(v.x) == 0 && sgn(v.y) == 0) return {u};
25     return {u - v, u + v};
26 }
27 vector<pii> convexLineIntersect(vector<Pt>& C, Line la) {
28     auto dis = [&](int p) {
29         return (la.b - la.a) ^ (C[p] - la.a);
30     };
31     auto gao = [&](int s) {
32         return cycTsearch(sz(C), [&](int i, int j) {
33             return sgn(dis(i) - dis(j)) == s;
34         });
35     };
36     int x = gao(1), y = gao(-1), n = sz(C);
37     if (sgn(dis(x)) < 0 || sgn(dis(y)) > 0) return {};
38     if (sgn(dis(x)) == 0 || sgn(dis(y)) == 0) {
39         int v = ((sgn(dis(x)) == 0 ? x : y) + n - 1) % n;
40         vector<pii> vec;
41         for (int i = 0; i < 3; i++, v = (v + 1) % n)
42             if (sgn(dis(v)) == 0) vec.emplace_back(v, v);
43         return vec;
44     }

```

```

45 auto get = [&](int l, int r, int s) {
46     while ((l + 1) % n != r) {
47         int m = ((l + r + (l < r ? 0 : n)) / 2) % n;
48         (sgn(dis(m)) == s ? l : r) = m;
49     }
50     if (sgn(dis(r)) == 0) return pii(r, r);
51     return pii(l, r);
52 };
53 return {get(x, y, 1), get(y, x, -1)};
54 }

```

11.15 Tangents

172759

```

1 // Dependency: cycTsearch(); T = double for circle tangents
2 auto circlePointTangent(Cir c, Pt p) {
3     vector<Line> res;
4     double dSq = abs2(p - c.o);
5     if (sgn(dSq - c.r * c.r) == 0) {
6         res.push_back({p, p + rot90(p - c.o)});
7     } else if (dSq > c.r * c.r) {
8         double s = dSq - c.r * c.r;
9         Pt v = p + (c.o - p) * s / dSq;
10        Pt u = rot90(c.o - p) * sqrt(s) * c.r / dSq;
11        res.push_back({p, v + u});
12        res.push_back({p, v - u});
13    }
14    return res;
15 }
16 auto circlesTangent(Cir c1, Cir c2, int sign1) {
17     // sign1 = 1 for outer tang, -1 for inner tang
18     vector<Line> res;
19     double dSq = abs2(c1.o - c2.o);
20     if (sgn(dSq) == 0) return res;
21     double d = sqrt(dSq);
22     Pt v = (c2.o - c1.o) / d;
23     double c = (c1.r - sign1 * c2.r) / d;
24     if (c * c > 1) return res;
25     double h = sqrt(max(0.0, 1.0 - c * c));
26     for (int sign2 = 1; sign2 >= -1; sign2 -= 2) {
27         Pt n = Pt(v.x * c - sign2 * h * v.y,
28                 v.y * c + sign2 * h * v.x);
29         Pt p1 = c1.o + n * c1.r;
30         Pt p2 = c2.o + n * (c2.r * sign1);
31         if (sgn(p1.x - p2.x) == 0 &&
32             sgn(p1.y - p2.y) == 0)
33             p2 = p1 + rot90(c2.o - c1.o);
34         res.push_back({p1, p2});
35     }
36     return res;
37 }
38 /* The point should be strictly out of hull
39    return arbitrary point on the tangent line */
40 pii pointConvexTangent(vector<Pt>& C, Pt p) {
41     auto gao = [&](int s) {
42         return cycTsearch(sz(C), [&](int x, int y) {
43             return ori(p, C[x], C[y]) == s;
44         });
45     };
46     return pii(gao(1), gao(-1));
47 } // return (a, b), ori(p, C[a], C[b]) >= 0

```

11.16 Bisector

70d32b

```

1 Pt bisector(Pt a, Pt b, Pt c) { Pt s = b-a, t = c-a;
2   t = t * abs(s) / abs(t); return s + t; }

```

11.17 Minimum Enclosing Circle

4fc473

```

1 // Dependency: circenter(); T = double
2 Cir minEnclosingCircle(vector<Pt> p) {
3     auto seed = chrono::steady_clock::now();
4     time_since_epoch().count();
5     shuffle(all(p), mt19937(seed));
6     double r = 0.0; Pt cent = p[0];
7     for (int i = 1; i < sz(p); i++) {
8         if (abs2(cent - p[i]) <= r) continue;
9         cent = p[i]; r = 0.0;
10        for (int j = 0; j < i; j++) {
11            if (abs2(cent - p[j]) <= r) continue;
12            cent = (p[i] + p[j]) / 2; r = abs2(p[j] - cent);
13            for (int k = 0; k < j; k++) {
14                if (abs2(cent - p[k]) <= r) continue;
15                cent = circenter(p[i], p[j], p[k]);
16                r = abs2(p[k] - cent);
17            }
18        }
19    }
20    return {cent, sqrt(r)};

```

11.18 Minkowski Sum

130078

```

1 // Dependency: convexHull()
2 auto minkowski(vector<Pt> a, vector<Pt> b) {
3     a = convexHull(a), b = convexHull(b);
4     int n = sz(a), m = sz(b);
5     vector<Pt> c = {a[0] + b[0]}, s1, s2;

```

```

6     for (int i = 0; i < n; i++) s1.pb(a[(i+1) % n] - a[i]);
7     for (int i = 0; i < m; i++) s2.pb(b[(i+1) % m] - b[i]);
8     for (int p1 = 0, p2 = 0; p1 < n || p2 < m;)
9         if (p2 == m ||
10            (p1 < n && sgn(s1[p1] ^ s2[p2]) >= 0))
11             c.pb(c.back() + s1[p1++]);
12         else
13             c.pb(c.back() + s2[p2++]);
14     return convexHull(c);
15 }

```

11.19 Convex Hull Distance

d820cb

```

1 // Dependency: minkowski(), inConvex(), pointSegDist()
2 // T = double
3 double convexHullDist(vector<Pt> A, vector<Pt> B) {
4     Pt O(0, 0); for (auto& p : B) p = 0 - p;
5     auto C = minkowski(A, B);
6     if (inConvex(C, O)) return 0;
7     double ans = pointSegDist(O, C.back(), C[0]);
8     for (int i = 0; i + 1 < sz(C); i++)
9         ans = min(ans, pointSegDist(O, C[i], C[i + 1]));
10    return ans;

```

11.20 Closest Point Pair

e7d4d5

```

1 // returns {min squared dist, {i, j}}, n >= 2, |coord| < 1e9
2 pair<T, pair<int, int>> closestPair(vector<Pt> p) {
3     vector<int> id(sz(p)); iota(all(id), 0);
4     sort(all(id),
5          [&](int a, int b) { return p[a].y < p[b].y; });
6     set<pair<Pt, int>> s; // (point, id)
7     pair<T, pair<int, int>> best = {(T)9e18, {-1, -1}};
8     int l = 0; for (int i : id) {
9         if (!best.first) break;
10        T d = sqrtl(best.first) + is_integral_v<T>;
11        while (p[id[l]].y <= p[i].y - d)
12            s.erase({p[id[l]], id[l]}), l++;
13        auto lo = s.lower_bound({p[i] - Pt{d, 0}, 0});
14        auto hi = s.upper_bound({p[i] + Pt{d, 0}, sz(p)});
15        for (; lo != hi; lo++) {
16            auto [q, j] = *lo;
17            best = min(best,
18                      {abs2(q - p[i]), {min(i, j), max(i, j)}});
19        } s.insert({p[i], i});
20    } return best;
21 }

```

11.21 Polygon-Circle Intersection

8d92c8

```

1 // T = double
2 double areaSector(Pt pa, Pt pb, double r) {
3     if (abs(pa) < abs(pb)) swap(pa, pb);
4     if (abs(pb) < eps) return 0;
5     double S, h, t;
6     double a = abs(pb), b = abs(pa), c = abs(pb - pa);
7     double cosB = pb * (pb - pa) / a / c;
8     double B = acos(cosB);
9     double cosC = pa * pb / a / b, C = acos(cosC);
10    if (a > r) {
11        S = C / 2 * r * r;
12        h = a * b * sin(C) / c;
13        if (h < r && B < PI / 2)
14            S -= acos(h / r) * r * r - h * sqrt(r * r - h * h);
15    } else if (b > r) {
16        t = PI - B - asin(sin(B) / r * a);
17        S = 0.5 * a * r * sin(t) + (C - t) / 2 * r * r;
18    } else {
19        S = 0.5 * sin(C) * a * b;
20    }
21    return S;
22 }

```

```

23 double areaPolyCircle(vector<Pt> poly, Pt O, double r) {
24     double S = 0;
25     int n = sz(poly);
26     for (int i = 0; i < n; i++)
27         S += areaSector(poly[i] - O,
28                         poly[(i + 1) % n] - O, r) *
29             ori(O, poly[i], poly[(i + 1) % n]);
30     return fabs(S);
31 }

```

11.22 Union of Polygons

a9896b

```

1 //Author:Gino, simple polygons any orientation, O((sum n)^2)
2 // T = ll: exact (|coord| <= 1e9); T = double: also works
3 double polysUnionArea(vector<vector<Pt>> poly) {
4     struct Fr { T p, q; }; // p / q, q > 0
5     auto lt = [](Fr a, Fr b) {
6         //return ( (__int128)a.p * b.q < ( __int128)b.p * a.q; // if T
6         // == long long
7         return a.p * b.q < b.p * a.q;
8     };
9     auto val = [&](Fr a) { // clamp to [0, 1], to double

```

```

10 if (lt(a, {0, 1})) return 0;
11 if (lt({1, 1}, a)) return 1;
12 return (double)a / a.q;
13 };
14 auto get = [&](int i, int e) {
15     return poly[i][(e + 1) % sz(poly[i])];
16 };
17 for (auto& p : poly)
18     if (dbarea(p) < 0) reverse(all(p));
19 double sum = 0;
20 for (int i = 0; i < sz(poly); i++)
21     for (int e = 0; e < sz(poly[i]); e++) {
22         Pt s = poly[i][e], t = get(i, e);
23         vector<pair<Fr, int>> c = {{0, 1}, 0}, {{1, 1}, 0}};
24         for (int j = 0; j < sz(poly); j++) {
25             if (i == j) continue;
26             for (int f = 0; f < sz(poly[j]); f++) {
27                 Pt a = poly[j][f], b = get(j, f);
28                 int ta = ori(s, t, a), tb = ori(s, t, b);
29                 if (!ta && !tb) {
30                     if (j < i && sgn((b-a) * (t-s)) > 0) {
31                         T d = abs2(t-s);
32                         c.pb({Fr{(a-s) * (t-s), d}, 1});
33                         c.pb({Fr{(b-s) * (t-s), d}, -1});
34                     }
35                     else if ((ta >= 0) != (tb >= 0)) {
36                         T x = (b-a) ^ (s-a), y = (b-a) ^ (t-a);
37                         if (x < y) x = -x, y = -y;
38                         c.pb({Fr{x, x-y}, ta >= 0 ? 1 : -1});
39                     }
40                 }
41             }
42             sort(all(c), [&](auto& u, auto& v)
43                 { return lt(u.first, v.first); });
44             double z = val(c[0].first), r = 0;
45             for (int k = 1, d = c[0].second; k < sz(c); k++) {
46                 double w = val(c[k].first);
47                 if (!d) r += w - z;
48                 d += c[k].second, z = w;
49             }
50             sum += (s^t) * r;
51         }
52     }
53     return sum / 2;
54 }

```

11.23 Rotating Sweep Line

1a5f20

```

1 // Dependency: ref()
2 struct Event {
3     Pt d; int u, v;
4     bool operator<(const Event& o) const {
5         return sgn(d ^ o.d) > 0;
6     };
7 void rotatingSweepLine(vector<Pt> pts) {
8     int n = sz(pts);
9     vector<int> ord(n), pos(n); vector<Event> e;
10    for (int i = 0; i < n; i++)
11        for (int j = i + 1; j < n; j++)
12            e.push_back({ref(pts[i] - pts[j]), i, j});
13    sort(all(e)); iota(all(ord), 0);
14    sort(all(ord), [&](int i, int j) {
15        return sgn(pts[i].y - pts[j].y) == 0 ?
16            pts[i].x < pts[j].x : pts[i].y < pts[j].y;
17    });
18    for (int i = 0; i < n; i++) pos[ord[i]] = i;
19    auto makeReverse = [](auto v) {
20        sort(all(v));
21        v.resize(unique(all(v)) - v.begin());
22        vector<pii> segs;
23        for (int i = 0, j = 0; i < sz(v); i = j) {
24            for (; j < sz(v) && v[j].l - v[i].l <= j - i; j++);
25            segs.emplace_back(v[i], v[j - 1] + 1 + 1);
26        }
27        return segs;
28    };
29    for (int i = 0, j = 0; i < sz(e); i = j) {
30        vector<int> tmp;
31        for (; j < sz(e) && !(e[i].l < e[j].l); j++)
32            tmp.pb(min(pos[e[i].u], pos[e[j].v]));
33        for (auto [l, r] : makeReverse(tmp)) {
34            reverse(ord.begin() + l, ord.begin() + r);
35            for (int t = l; t < r; t++) pos[ord[t]] = t;
36            // update value here
37        }
38    }
39 }

```

11.24 Pick's Theorem

Consider a polygon which vertices are all lattice points.
 Let i = number of points inside the polygon.
 Let b = number of points on the boundary of the polygon.
 Then we have the following formula:

$$\text{Area} = i + b/2 - 1$$

12 Modular

12.1 Basic

032b2b

```

1 const ll mod = 998244353;
2 ll add(ll a, ll b) { return (a += b) >= mod ? a - mod : a; }
3 ll sub(ll a, ll b) { return (a -= b) < 0 ? a + mod : a; }
4 ll mul(ll a, ll b) { return a * b % mod; }
5 ll Pow(ll b, ll e) { ll r = 1;
6     for (; e; e >>= 1, b = mul(b, b)) if (e & 1) r = mul(r, b);
7     return r; }

```

12.2 Mod Sum / Floor Sum

9081a9

```

1 // modSum(to, c, k, m) sums (k*i+c) mod m for i < to.
2 // divSum computes the corresponding sum of floor((k*i+c)/m).
3 // Both run in O(log m).
4 using ull = unsigned long long;
5
6 ull sumsq(ull to) { return to / 2 * ((to - 1) | 1); }
7
8 ull divSum(ull to, ull c, ull k, ull m) {
9     ull res = k / m * sumsq(to) + c / m * to;
10    k %= m;
11    c %= m;
12    if (!k) return res;
13    ull to2 = (to * k + c) / m;
14    return res + (to - 1) * to2
15        - divSum(to2, m - 1 - c, m, k);
16 }
17
18 ll modSum(ull to, ll c, ll k, ll m) {
19     c = ((c % m) + m) % m;
20     k = ((k % m) + m) % m;
21     return to * c + k * sumsq(to) - m * divSum(to, c, k, m);
22 }

```

12.3 Mod Sqrt

670f74

```

1 // sqrtMod(a, p) finds x with x^2 = a (mod p).
2 // Tonelli-Shanks runs in O(log^2 p), usually O(log p).
3
4 // Pow(b,e,p): 3-arg overload
5 ll Pow(ll b, ll e, ll p) {
6     ll r = 1;
7     for (; e; e >>= 1, b = b * b % p)
8         if (e & 1) r = r * b % p;
9     return r;
10 }
11
12 ll sqrtMod(ll a, ll p) {
13     a %= p;
14     if (a < 0) a += p;
15     if (a == 0) return 0;
16     assert(Pow(a, (p - 1) / 2, p) == 1);
17     if (p % 4 == 3) return Pow(a, (p + 1) / 4, p);
18     ll s = p - 1, n = 2;
19     int r = 0, m;
20     while (s % 2 == 0) r++, s /= 2;
21     while (Pow(n, (p - 1) / 2, p) != p - 1) n++;
22     ll x = Pow(a, (s + 1) / 2, p);
23     ll b = Pow(a, s, p), g = Pow(n, s, p);
24     for (; r = m) {
25         ll t = b;
26         for (m = 0; m < r && t != 1; m++) t = t * t % p;
27         if (m == 0) return x;
28         ll gs = Pow(g, 1LL << (r - m - 1), p);
29         g = gs * gs % p;
30         x = x * gs % p;
31         b = b * g % p;
32     }
33 }

```

12.4 BSGS / exBSGS

9b8b0f

```

1 // rep(i,a,b) here is INCLUSIVE: for (i = a; i <= b; i++)
2 i64 BSGS(i64 X, i64 Y, i64 P) {
3     X %= P, Y %= P;
4     assert(gcd(X, P) == 1);
5     const i64 B = sqrtl(P) + 1;
6     unordered_map<i64, int> mp;
7     for (i64 i = 0, cur = Y; i <= B; i++) {
8         mp[cur] = i;
9         cur = cur * X % P;
10    }
11    i64 step = 1;
12    rep(i, 1, B) step = step * X % P;
13    for (i64 p = 1, cur = 1; p <= B; p++) {
14        cur = cur * step % P;
15        if (mp.contains(cur)) {
16            return B * p - mp[cur];
17        }
18    }
19 }

```

```

18 }
19 return -1;
20 }
21
22 i64 exBSGS(i64 X, i64 Y, i64 P) {
23     X %= P, Y %= P;
24     i64 D = 1;
25     int add = 0;
26     while (true) {
27         const auto g = gcd(X, P);
28         if (g == 1) break;
29         if (D == Y) return add;
30         if (Y % g != 0) return -1;
31         Y /= g;
32         P /= g;
33         add++;
34         D = D * (X / g) % P;
35     }
36     const i64 B = sqrtl(P) + 1;
37     unordered_map<i64, int> mp;
38     for (i64 i = 0, cur = Y; i <= B; i++) {
39         mp[cur] = i;
40         cur = cur * X % P;
41     }
42     i64 step = 1;
43     rep (i, 1, B) step = step * X % P;
44     for (i64 p = 1, cur = D; p <= B; p++) {
45         cur = cur * step % P;
46         if (mp.contains(cur)) {
47             return B * p - mp[cur] + add;
48         }
49     }
50     return -1;
51 }

```

13 Number Theory

13.1 Miller Rabin

2502ec

```

1 // n <= 7.2e18 deterministic; 1e6 calls ~0.26s
2 using ull = unsigned long long;
3
4 ull modmul(ull a, ull b, ull M) {
5     ll ret = a * b - M * ull(1.L / M * a * b);
6     return ret + M * (ret < 0) - M * (ret >= (ll)M);
7 }
8
9 ull modpow(ull b, ull e, ull mod) {
10     ull ans = 1;
11     for (; e; b = modmul(b, b, mod), e /= 2)
12         if (e & 1) ans = modmul(ans, b, mod);
13     return ans;
14 }
15
16 bool isPrime(ull n) {
17     if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
18     ull A[] = {
19         2, 325, 9375, 28178, 450775, 9780504, 1795265022
20     };
21     ull s = __builtin_ctzll(n - 1), d = n >> s;
22     for (ull a : A) {
23         ull p = modpow(a % n, d, n), i = s;
24         while (p != 1 && p != n - 1 && a % n && i--)
25             p = modmul(p, p, n);
26         if (p != n - 1 && i != s) return 0;
27     }
28     return 1;
29 }

```

13.2 Pollard Rho

d819f6

```

1 // factor(n): prime factors with multiplicity
2 // 1e18 semiprime ~0.65ms; needs: MillerRabin
3
4 ull pollard(ull n) {
5     ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
6     auto f = [&](ull x) { return modmul(x, x, n) + i; };
7     while (t++ % 40 || __gcd(prd, n) == 1) {
8         if (x == y) x = ++i, y = f(x);
9         if ((q = modmul(prd, max(x, y) - min(x, y), n)) prd =
10             q;
11             x = f(x);
12             y = f(f(y));
13         }
14     }
15     return __gcd(prd, n);
16 }
17
18 vector<ull> factor(ull n) {
19     if (n == 1) return {};
20     if (isPrime(n)) return {n};
21     ull x = pollard(n);
22     auto l = factor(x), r = factor(n / x);

```

```

21 l.insert(l.end(), all(r));
22 return l;
23 }

```

13.3 Linear Sieve

$O(n)$, each i crossed out once by its least prime p :

```

for i in 2..n:
    if lpf[i] == 0: lpf[i] = i, primes.pb(i), f[i] = f(p)
    for p in primes:
        if p > lpf[i] or i * p > n: break
        lpf[i*p] = p
        f[i*p] = (i % p ? f[i] * f[p] : <recurrence>)

```

Recurrence for ip when $p \mid i$ (let $i = p^k m, p \nmid m$):

- $\varphi(ip) = \varphi(i) \cdot p, \quad \mu(ip) = 0, \quad \sigma_0(ip) = \sigma_0(i) \cdot \frac{k+2}{k+1}$
- General: keep $pw[i] = p^k; f(ip) = f\left(\frac{i}{pw[i]}\right) \cdot f(pw[i] \cdot p)$
- $\varphi(ip) = \varphi(i) \cdot (p - [p \nmid i])$ (one formula for both cases)

Euler phi table alone (no primes): $O(n \log \log n)$

for i : $\phi[i]=i$; for p prime: for $j=p, 2p, \dots$: $\phi[j] -= \phi[j]/p$

13.4 Lucy (prime counting / prime sum)

22d982

```

1 template<class T, class F>
2 pair<vector<T>, vector<T>> Lucy(const i64 N, const F f) {
3     const i64 sqrtN = sqrtl(N);
4     vector<T> Sl(sqrtN + 1);
5     vector<T> Sh(sqrtN + 1);
6     for (int i = 1; i <= sqrtN; i++) {
7         Sl[i] = f(i) - 1;
8         Sh[i] = f(f64(N) / i) - 1;
9     }
10    for (const int &p : primes) {
11        const i64 pp = i64(p) * p;
12        if (pp > N) break;
13        const int R = min(sqrtN, N / pp);
14        const int M = sqrtN / p;
15        const auto pi = Sl[p - 1];
16        const auto fp = Sl[p] - Sl[p - 1];
17        for (int i = 1; i <= M; i++) Sh[i] -= fp * (Sh[i * p] -
18            pi);
19        for (int i = M + 1; i <= R; i++) Sh[i] -= fp * (Sl[N /
20            (f64(i) * p)] - pi);
21        for (int i = sqrtN; i >= pp; i--) Sl[i] -= fp * (Sl[i /
22            p] - pi);
23    }
24    return {Sl, Sh};
25 }

```

13.5 Black (Min25 sieve)

225afd

```

1 template<class T, class F1, class F2>
2 T Black(const i64 N, const F1 f_prime, const F2 f_k) {
3     const i64 sqrtN = sqrtl(N);
4     const int np = int(size(primes));
5     T res = 1 + f_prime(N);
6     function<void(i64, i64, i64, T, T)> rec = [&](i64 t, i64
7         i, i64 c, T ft, T fu) {
8         const auto f_next = fu * f_k(primes[i], c + 1);
9         res += f_next + ft * (f_prime(f64(N) / t) -
10             f_prime(primes[i]));
11         const i64 lim = sqrtl(f64(N) / t);
12         if (primes[i] <= lim) {
13             rec(t * primes[i], i, c + 1, f_next, fu);
14         }
15         for (int j = i + 1; j < np; j++) {
16             if (primes[j] > lim) break;
17             rec(t * primes[j], j, 1, ft * f_k(primes[j], 1), ft);
18         }
19     };
20     for (int i = 0; i < np; i++) {
21         if (primes[i] <= sqrtN) {
22             rec(primes[i], i, 1, f_k(primes[i], 1), 1);
23         }
24     }
25     return res;
26 }

```

13.6 Fast GCD

797563

```

1 template <size_t N, size_t SQRT>
2 struct GCD {
3     int sq, minp[N], gcd_table[SQRT][SQRT];
4     vector<int> primes;
5     array<int, 3> fac[N];
6     void init(const int n) {
7         sq = int(sqrt(n));
8         for (int i = 0; i <= sq; ++i) {
9             gcd_table[0][i] = gcd_table[i][0] = i;
10        }
11        for (int i = 1; i <= sq; ++i) {
12            for (int j = 1; j <= i; ++j) {

```

```

13 gcd_table[i][j] = gcd_table[j][i] = gcd_table[j][i %
14 j];
15 }
16 for (int i = 1; i <= n; ++i) {
17 minp[i] = i;
18 }
19 fac[1] = {1, 1, 1};
20 for (int i = 2; i <= n; ++i) {
21 if (minp[i] == i) {
22 primes.push_back(i);
23 fac[i] = {1, 1, i};
24 }
25 for (const int &p : primes) {
26 if (i * p > n) break;
27 const int j = i * p;
28 minp[j] = p;
29 fac[j] = fac[i];
30 fac[j][0] *= p;
31 ranges::sort(fac[j]);
32 if (i % p == 0) break;
33 }
34 }
35 }
36 int fastGCD(int x, int y) {
37 int g = 1;
38 for (const int &f : fac[x]) {
39 int c = 1;
40 if (f <= sq) {
41 c = gcd_table[f][y % f];
42 } else if (y % f == 0) {
43 c = f;
44 }
45 y /= c;
46 g *= c;
47 }
48 return g;
49 }
50 };

```

13.7 CRT

3721bc

```

1 // Solves x = a (mod m), x = b (mod n).
2 // m,n need not be coprime; asserts if no solution.
3 // Needs m*n < 2^62-ish; extgcd returns a*x+b*y=g.
4 ll extgcd(ll a, ll b, ll& x, ll& y) {
5 if (!b) return x = 1, y = 0, a;
6 ll g = extgcd(b, a % b, y, x);
7 return y -= a / b * x, g;
8 }
9
10 ll crt(ll a, ll m, ll b, ll n) {
11 if (n > m) swap(a, b), swap(m, n);
12 ll x, y, g = extgcd(m, n, x, y);
13 assert((a - b) % g == 0);
14 x = (b - a) % n * x % n / g * m + a;
15 return x < 0 ? x + m * n / g : x;
16 }

```

13.8 Extended Lucas

18001d

```

1 typedef long long ll;
2 void exgcd(ll a, ll b, ll& x, ll& y) {
3 if (!b)
4 x = 1, y = 0;
5 else
6 exgcd(b, a % b, y, x), y -= a / b * x;
7 }
8 ll inv(ll a, ll p) {
9 ll x, y;
10 exgcd(a, p, x, y);
11 return (x + p) % p;
12 }
13 ll qpow(ll a, ll b, ll p) {
14 ll r = 1;
15 for (; b >= 1; a = a * a % p)
16 if (b & 1) r = r * a % p;
17 return r;
18 }
19 ll CRT(int n, ll* a, ll* m) {
20 ll M = 1, p = 0;
21 for (int i = 1; i <= n; ++i) M = M * m[i];
22 for (int i = 1; i <= n; ++i) {
23 ll w = M / m[i], x, y;
24 exgcd(w, m[i], x, y);
25 p = (p + a[i] * w * x % M) % M;
26 }
27 return (p % M + M) % M;
28 }
29 ll calc(ll n, ll x, ll P) {
30 if (!n) return 1;
31 ll s = 1;

```

```

32 for (ll i = 1; i <= P; i++)
33 if (i % x) s = s * i % P;
34 s = qpow(s, n / P, P);
35 for (ll i = n / P * P + 1; i <= n; ++i)
36 if (i % x) s = i % P * s % P;
37 return s * calc(n / x, x, P) % P;
38 }
39 ll multilucas(ll n, ll m, ll x, ll P) {
40 if (n < m) return 0;
41 int cnt = 0;
42 for (ll i = n; i; i /= x) cnt += i / x;
43 for (ll i = m; i; i /= x) cnt -= i / x;
44 for (ll i = n - m; i; i /= x) cnt -= i / x;
45 return qpow(x, cnt, P) % P * calc(n, x, P) % P *
46 inv(calc(m, x, P), P) % P *
47 inv(calc(n - m, x, P), P) % P;
48 }
49 ll exlucas(ll n, ll m, ll P) {
50 int cnt = 0;
51 ll p[20], a[20];
52 for (ll i = 2; i * i <= P; i++) {
53 if (P % i == 0) {
54 p[++cnt] = 1;
55 while (P % i == 0) p[cnt] = p[cnt] * i, P /= i;
56 a[cnt] = multilucas(n, m, i, p[cnt]);
57 }
58 if (P > 1) p[++cnt] = P, a[cnt] = multilucas(n, m, P, P);
59 return CRT(cnt, a, p);
60 }

```

13.9 Divisor Block

3ae3d4

```

1 // calls f(l, r, q) for each maximal [l, r] with floor(n/i)
2 = q
3 void divisorBlock(ll n, auto f) {
4 for (ll l = 1, r, q; l <= n; l = r + 1) {
5 q = n / l, r = n / q;
6 f(l, r, q);
7 }

```

13.10 Formulas

- Pisano Period: 任何線性遞迴（比如費氏數列）模任何一個數字 M 都會循環，找循環節 $\pi(M)$ 先質因數分解 $M = \prod p_i^{e_i}$ ，然後 $\pi(M) = \text{lcm}(\pi(p_i^{e_i}))$
- Inversion: $aa^{-1} \equiv 1 \pmod{m}$, exists iff $\gcd(a, m) = 1$. Linear: $a^{-1} \equiv (m - \lfloor \frac{m}{a} \rfloor) \times (m \bmod a)^{-1} \pmod{m}$
- Fermat: $a^p \equiv a \pmod{p}$; Euler: $a^{\varphi(n)} \equiv 1 \pmod{n}$ if $\gcd(a, n) = 1$. Not coprime: 分解 $n = \prod p_i^{e_i}$ ，對每個 $p_i^{e_i}$ 分開看（互質：Euler / 不互質：指數 $\geq e_i$ 直接是 0），最後 CRT 合併。Also $a^b \equiv a^{b \bmod \varphi(n) + \varphi(n)} \pmod{n}$ for $b \geq \log_2 n$.
- Wilson: $(p-1)! \equiv -1 \pmod{p}$
- $\varphi(n) = n \prod_{p|n} (1 - \frac{1}{p})$, $\sum_{d|n} \varphi(d) = n$
- Divisor function: $n = \prod p_i^{a_i}$, $\sigma_x(n) = \prod \frac{p_i^{(a_i+1)x} - 1}{p_i^x - 1}$ ($x \neq 0$), $\sigma_0(n) = \prod (a_i + 1)$
- CRT (coprime): $M = \prod m_i$, $M_i = \frac{M}{m_i}$, $x = \sum a_i M_i (M_i^{-1} \bmod m_i) \pmod{M}$
General: $x = m_1 p + a_1 = m_2 q + a_2 \Rightarrow m_1 p - m_2 q = a_2 - a_1$, solve by extgcd, answer mod $\text{lcm}(m_1, m_2)$
- Avoiding overflow: $ca \bmod cb = c(a \bmod b)$
- Dirichlet convolution: $(f * g)(n) = \sum_{d|n} f(d)g(\frac{n}{d})$. $\varepsilon(n) = [n=1]$, $1(n) = 1$, $id(n) = n$; $\mu * 1 = \varepsilon$, $\varphi = \mu * id$, $id = \varphi * 1$, $\sigma_0 = 1 * 1$, $\sigma_1 = id * 1$
- $[\gcd(a, b) = 1] = \sum_{d|\gcd(a,b)} \mu(d)$; Möbius inversion: $f = g * 1 \Leftrightarrow g = f * \mu$
- $\sum_{i=1}^n \sum_{j=1}^m [\gcd(i, j) = 1] = \sum_d \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$ (divisor block, $O(\sqrt{n})$ per query)
- Legendre: $v_p(n!) = \sum_{k \geq 1} \lfloor \frac{n}{p^k} \rfloor = \frac{n - s_p(n)}{p-1}$, s_p = digit sum in base p
- Lucas (p prime): $\binom{n}{m} \equiv \prod_i \binom{n_i}{m_i} \pmod{p}$ over base- p digits; Kummer: $v_p(\binom{n}{m})$ = number of carries adding $m + (n - m)$ in base p

14 Polynomial

14.1 Generating Functions

OGF $A(x) = \sum a_i x^i$: $A(rx) \Rightarrow r^n a_n$; $AB \Rightarrow \sum_i a_i b_{n-i}$; $A^k \Rightarrow \sum_{i_1+\dots+i_k=n} a_{i_1} \dots a_{i_k}$; $xA'(x) \Rightarrow na_n$; $\frac{A(x)}{1-x} \Rightarrow \sum_{i \leq n} a_i$.

EGF $A(x) = \sum a_i \frac{x^i}{i!}$: $A^{(k)}(x) \Rightarrow a_{n+k}$; $AB \Rightarrow \sum_i \binom{n}{i} a_i b_{n-i}$; $A^k \Rightarrow \sum_{i_1+\dots+i_k=n} \frac{n!}{i_1! \dots i_k!} a_{i_1} \dots a_{i_k}$; $xA' \Rightarrow na_n$.

$(1+x)^n = \sum \binom{n}{i} x^i$, $\frac{1}{(1-x)^n} = \sum_i \binom{i+n-1}{n-1} x^i$, $\frac{1}{1-x} = \sum x^i$, $e^x = \sum \frac{x^i}{i!}$, $-\ln(1-x) = \sum_{i \geq 1} \frac{x^i}{i}$

Catalan $C(x) = \frac{1-\sqrt{1-4x}}{2x}$; Fibonacci $\frac{x}{1-x-x^2}$; $\sum_i ix^i = \frac{x}{(1-x)^2}$; partitions $\prod_k \frac{1}{1-x^k}$.

14.2 NTT

131fe6

```
1 // mod 998244353; 1e9+7 is NOT ntt-friendly
2 const int N = 1 << 20, G = 3;
3 using Poly = vector<ll>;
4
5 struct NTT {
6     ll w[N];
7     NTT() {
8         ll dw = Pow(G, (mod - 1) / N);
9         w[0] = 1;
10        for (int i = 1; i < N; i++)
11            w[i] = mul(w[i - 1], dw);
12    }
13    void operator()(Poly& a, bool inv = false) {
14        int n = sz(a);
15        for (int j = 1, x = 0; j < n - 1; j++) {
16            for (int k = n >> 1; (x ^= k) < k; k >>= 1);
17            if (j < x) swap(a[j], a[x]);
18        }
19        for (int L = 2; L <= n; L <= 1) {
20            int dx = N / L, dl = L >> 1;
21            for (int i = 0; i < n; i += L)
22                for (int j = i, x = 0; j < i + dl; j++, x += dx) {
23                    ll tmp = mul(a[j + dl], w[x]);
24                    a[j + dl] = sub(a[j], tmp);
25                    a[j] = add(a[j], tmp);
26                }
27        }
28        if (inv) {
29            reverse(1 + all(a));
30            ll invn = Pow(n, mod - 2);
31            for (ll& x : a) x = mul(x, invn);
32        }
33    } ntt;
34
35    // polyMul: a*b truncated to bound terms, O(n log n).
36    // Result length must be <= N.
37    Poly polyMul(Poly a, Poly b, int bound = N) {
38        int m = sz(a) + sz(b) - 1, n = 1;
39        while (n < m) n <= 1;
40        a.resize(n), b.resize(n);
41        ntt(a), ntt(b);
42        for (int i = 0; i < n; i++) a[i] = mul(a[i], b[i]);
43        ntt(a, true);
44        a.resize(min(m, bound));
45        return a;
46    }
47 }
```

NTT primes (root): 998244353 (3), 167772161 (3), 469762049 (3), 1004535809 (3), 2013265921 (31), 1224736769 (3), 7340033 (3), 65537 (3); 985661441(3), 2281701377(3) (32-bit unsigned), 194555039024054273(5), 4179340454199820289(3) (64-bit).

14.3 FFT

5568a8

```
1 // Rounding is safe if
2 // (sum a[i]^2 + sum b[i]^2) log2(N) < 9e14.
3 // In practice, 1e16 or higher often works.
4 using C = complex<double>;
5 using vd = vector<double>;
6
7 void fft(vector<C>& a) {
8     int n = sz(a), L = 31 - __builtin_clz(n);
9     static vector<complex<long double>> R(2, 1);
10    static vector<C> rt(2, 1);
11    for (static int k = 2; k < n; k *= 2) {
12        R.resize(n), rt.resize(n);
13        auto x = polar(1.0L, acos(-1.0L) / k);
14        for (int i = k; i < 2 * k; i++)
15            rt[i] = R[i] = i & 1 ? R[i / 2] * x : R[i / 2];
16    }
17    vi rev(n);
18    for (int i = 0; i < n; i++)
19        rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
20    for (int i = 0; i < n; i++)
21        if (i < rev[i]) swap(a[i], a[rev[i]]);
22    for (int k = 1; k < n; k *= 2)
23        for (int i = 0; i < n; i += 2 * k)
24            for (int j = 0; j < k; j++) {
25                auto* x = (double*)&rt[j + k];
26                auto* y = (double*)&a[i + j + k];
27                C z(x[0] * y[0] - x[1] * y[1],
28                  x[0] * y[1] + x[1] * y[0]);
29                a[i + j + k] = a[i + j] - z;
30                a[i + j] += z;
31            }
```

```
32 }
33
34 vd convFFT(const vd& a, const vd& b) {
35     if (a.empty() || b.empty()) return {};
36     vd res(sz(a) + sz(b) - 1);
37     int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
38     vector<C> in(n), out(n);
39     copy(all(a), begin(in));
40     for (int i = 0; i < sz(b); i++) in[i].imag(b[i]);
41     fft(in);
42     for (C& x : in) x *= x;
43     for (int i = 0; i < n; i++)
44         out[i] = in[-i & (n - 1)] - conj(in[i]);
45     fft(out);
46     for (int i = 0; i < sz(res); i++)
47         res[i] = imag(out[i]) / (4 * n);
48     return res;
49 }
```

14.4 FFT Mod (arbitrary modulus)

1448dd

```
1 // Higher precision FFT convolution modulo M.
2 // Safe when N log2(N) M < 8.6e14.
3 // Inputs are in [0,M), with M < 2^31-ish.
4 // needs: FFT
5 vector<ll> convMod(const vector<ll>& a,
6                   const vector<ll>& b, ll M) {
7     if (a.empty() || b.empty()) return {};
8     vector<ll> res(sz(a) + sz(b) - 1);
9     int B = 32 - __builtin_clz(sz(res)), n = 1 << B;
10    int cut = int(sqrt(M));
11    vector<C> L(n), R(n), outs(n), outl(n);
12    for (int i = 0; i < sz(a); i++)
13        L[i] = C((int)(a[i] / cut), (int)(a[i] % cut));
14    for (int i = 0; i < sz(b); i++)
15        R[i] = C((int)(b[i] / cut), (int)(b[i] % cut));
16    fft(L), fft(R);
17    for (int i = 0; i < n; i++) {
18        int j = -i & (n - 1);
19        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
20        outs[j] = (L[i] - conj(L[j])) * R[i]
21                  / (2.0 * n * C(0, 1));
22    }
23    fft(outl), fft(outs);
24    for (int i = 0; i < sz(res); i++) {
25        ll av = ll(real(outl[i]) + .5);
26        ll cv = ll(imag(outs[i]) + .5);
27        ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i]) + .5);
28        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
29    }
30    return res;
31 }
```

14.5 FWHT

f16f34

```
1 using Poly = vector<ll>;
2
3 // op: 0 = AND, 1 = OR, 2 = XOR; inv = inverse transform
4 // conv: fwt(a,op), fwt(b,op), a[i] = mul(a[i], b[i]),
5 // fwt(a,op,1)
6 void fwt(Poly& a, int op, bool inv = false) {
7     int n = sz(a);
8     ll i2 = (mod + 1) / 2;
9     for (int s = 1; s < n; s <= 1)
10        for (int i = 0; i < n; i += s << 1)
11            for (int j = i; j < i + s; j++) {
12                ll &x = a[j], &y = a[j + s];
13                if (op == 0) x = inv ? sub(x, y) : add(x, y);
14                else if (op == 1) y = inv ? sub(y, x) : add(y, x);
15                else {
16                    tie(x, y) = pair(add(x, y), sub(x, y));
17                    if (inv) x = mul(x, i2), y = mul(y, i2);
18                }
19            }
20 }
```

14.6 Polynomial Operations

00d95f

```
1 // needs: NTT
2 // needs: ModSqrt
3 // polyInv(a): inverse modulo x^n, a[0] != 0.
4 Poly polyInv(Poly a) {
5     int n = sz(a);
6     Poly res(1, Pow(a[0], mod - 2));
7     for (int m = 1; m < n; m <= 1) {
8         if (n < m * 2) a.resize(m * 2);
9         Poly v1(a.begin(), a.begin() + m * 2), v2 = res;
10        v1.resize(m * 4), v2.resize(m * 4);
11        ntt(v1), ntt(v2);
12        for (int i = 0; i < m * 4; i++)
13            v1[i] = mul(mul(v1[i], v2[i]), v2[i]);
14        ntt(v1, true);
15        res.resize(m * 2);
16    }
```

```

16     for (int i = 0; i < m; i++) res[i] = add(res[i],
res[i]);
17     for (int i = 0; i < m * 2; i++)
18         res[i] = sub(res[i], vl[i]);
19 }
20 res.resize(n);
21 return res;
22 }
23
24 // polyDiv(a,b): a = bQ + R, b.back() != 0.
25 pair<Poly, Poly> polyDiv(Poly a, Poly b) {
26     int n = sz(a), m = sz(b), k = n - m + 1;
27     if (n < m) return {{0}, a};
28     Poly ra = a, rb = b;
29     reverse(all(ra)), ra.resize(k);
30     reverse(all(rb)), rb.resize(k);
31     Poly q = polyMul(ra, polyInv(rb), k);
32     reverse(all(q));
33     Poly res = polyMul(b, q), r(m - 1);
34     for (int i = 0; i < m - 1; i++) r[i] = sub(a[i], res[i]);
35     return {q, r};
36 }
37
38 // polySqrt(a): square root modulo x^n; asserts if a[0]
39 // not a residue.
40 Poly polySqrt(Poly a) {
41     if (a.empty()) return {0};
42     int n = sz(a), m = 0;
43     while (m < n && a[m] == 0) m++;
44     if (m == n) return Poly(n);
45     if (m & 1) return {-1};
46     a.erase(a.begin(), a.begin() + m);
47     ll z = sqrtMod(a[0], mod);
48     Poly q(1, z);
49     ll inv2 = (mod + 1) / 2;
50     for (int k = 1; k < n - m + 1; k <= 1) {
51         if (n - m < k * 2) a.resize(k * 2);
52         q.resize(k * 2);
53         Poly f2 = polyMul(q, q, k * 2);
54         for (int i = 0; i < k * 2; i++) f2[i] = sub(f2[i],
a[i]);
55         f2 = polyMul(f2, polyInv(q), k * 2);
56         for (int i = 0; i < k * 2; i++)
57             q[i] = sub(q[i], mul(f2[i], inv2));
58     }
59     q.resize(n - m);
60     Poly res(n);
61     for (int i = 0; i < sz(q); i++) res[i + m / 2] = q[i];
62     return res;
63 }
64
65 Poly deriv(Poly a) {
66     int n = sz(a);
67     Poly res(max(0, n - 1));
68     for (int i = 0; i < n - 1; i++)
69         res[i] = mul(a[i + 1], i + 1);
70     return res;
71 }
72
73 Poly integ(Poly a) {
74     int n = sz(a);
75     Poly res(n + 1);
76     for (int i = 0; i < n; i++)
77         res[i + 1] = mul(a[i], Pow(i + 1, mod - 2));
78     return res;
79 }
80
81 // polyLn(a): formal logarithm, a[0] = 1.
82 Poly polyLn(Poly a) {
83     int n = sz(a);
84     if (n == 1) return {0};
85     Poly d = deriv(a);
86     a.pop_back();
87     return integ(polyMul(d, polyInv(a), n - 1));
88 }
89
90 // polyExp(a): formal exponential, a[0] = 0.
91 Poly polyExp(Poly a) {
92     int n = sz(a);
93     Poly q(1, 1);
94     a[0] = add(a[0], 1);
95     for (int m = 1; m < n; m <= 1) {
96         if (n < m * 2) a.resize(m * 2);
97         Poly g(a.begin(), a.begin() + m * 2), h = q;
98         h.resize(m * 2), h = polyLn(h);
99         for (int i = 0; i < m * 2; i++) g[i] = sub(g[i], h[i]);
100        q = polyMul(g, q, m * 2);
101    }
102    q.resize(n);

```

```

103    return q;
104 }
105
106 Poly polyPow(Poly a, ll k) {
107     int n = sz(a), m = 0;
108     Poly ans(n);
109     if (!k) return ans[0] = 1, ans;
110     while (m < n && a[m] == 0) m++;
111     if (k && m && (k >= n || k * m >= n)) return ans;
112     if (m == n) return ans[0] = 1, ans;
113     int lead = m * k;
114     Poly b(a.begin() + m, a.end());
115     ll base = Pow(b[0], k), inv = Pow(b[0], mod - 2);
116     for (int i = 0; i < n - m; i++) b[i] = mul(b[i], inv);
117     b = polyLn(b);
118     for (ll& x : b) x = mul(x, k % mod);
119     b = polyExp(b);
120     for (int i = lead; i < n; i++)
121         ans[i] = mul(b[i - lead], base);
122     return ans;
123 }
124
125 vector<ll> evaluate(Poly a, vector<ll> x) {
126     if (x.empty()) return {};
127     int n = sz(x);
128     vector<Poly> up(n * 2);
129     for (int i = 0; i < n; i++) up[i + n] = {sub(0, x[i]), 1};
130     for (int i = n - 1; i > 0; i--)
131         up[i] = polyMul(up[i * 2], up[i * 2 + 1]);
132     vector<Poly> down(n * 2);
133     down[1] = polyDiv(a, up[1]).second;
134     for (int i = 2; i < n * 2; i++)
135         down[i] = polyDiv(down[i >> 1], up[i]).second;
136     vector<ll> y(n);
137     for (int i = 0; i < n; i++) y[i] = down[i + n][0];
138     return y;
139 }
140
141 Poly interpolate(vector<ll> x, vector<ll> y) {
142     int n = sz(x);
143     vector<Poly> up(n * 2);
144     for (int i = 0; i < n; i++) up[i + n] = {sub(0, x[i]), 1};
145     for (int i = n - 1; i > 0; i--)
146         up[i] = polyMul(up[i * 2], up[i * 2 + 1]);
147     Poly a = evaluate(deriv(up[1]), x);
148     for (int i = 0; i < n; i++)
149         a[i] = mul(y[i], Pow(a[i], mod - 2));
150     vector<Poly> down(n * 2);
151     for (int i = 0; i < n; i++) down[i + n] = {a[i]};
152     for (int i = n - 1; i > 0; i--) {
153         Poly lhs = polyMul(down[i * 2], up[i * 2 + 1]);
154         Poly rhs = polyMul(down[i * 2 + 1], up[i * 2]);
155         down[i].resize(sz(lhs));
156         for (int j = 0; j < sz(lhs); j++)
157             down[i][j] = add(lhs[j], rhs[j]);
158     }
159     return down[1];
160 }

```

14.7 Berlekamp Massey

39b0fd

```

1 // Returns c with s[i] = sum c[j]*s[i-1-j].
2 // O(n^2); provide at least 2k samples for order k.
3 using Poly = vector<ll>;
4
5 Poly berlekampMassey(Poly s) {
6     int n = sz(s), L = 0, m = 0;
7     Poly C(n), B(n), T;
8     C[0] = B[0] = 1;
9     ll b = 1;
10    for (int i = 0; i < n; i++) {
11        m++;
12        ll d = s[i] % mod;
13        for (int j = 1; j <= L; j++)
14            d = add(d, mul(C[j], s[i - j]));
15        if (!d) continue;
16        T = C;
17        ll coef = mul(d, Pow(b, mod - 2));
18        for (int j = m; j < n; j++)
19            C[j] = sub(C[j], mul(coef, B[j - m]));
20        if (2 * L > i) continue;
21        L = i + 1 - L;
22        B = T, b = d, m = 0;
23    }
24    C.resize(L + 1);
25    C.erase(C.begin());
26    for (ll& x : C) x = x ? mod - x : 0;
27    return C;
28 }

```

14.8 Linear Recurrence (Kitamasa)

92237b

```

1// k-th term of a recurrence in  $O(n^2 \log k)$ , mod  $< 2^{31}$ .
2// Example: linearRec({0,1},{1,1},k) returns Fib(k).
3using Poly = vector<ll>;
4
5ll linearRec(Poly S, Poly tr, ll k) {
6    int n = sz(tr);
7    auto combine = [&](Poly a, Poly b) {
8        Poly res(n * 2 + 1);
9        for (int i = 0; i <= n; i++)
10            for (int j = 0; j <= n; j++)
11                res[i + j] = add(res[i + j], mul(a[i], b[j]));
12        for (int i = 2 * n; i > n; i--)
13            for (int j = 0; j < n; j++)
14                res[i - 1 - j] = add(res[i - 1 - j],
15                                     mul(res[i], tr[j]));
16        res.resize(n + 1);
17        return res;
18    };
19    Poly pol(n + 1, e(pol));
20    pol[0] = e[1] = 1;
21    for (++k; k % 2) pol = combine(pol, e);
22    e = combine(e, e);
23    ll res = 0;
24    for (int i = 0; i < n; i++)
25        res = add(res, mul(pol[i + 1], S[i]));
26    return res;
27}

```

14.9 Lagrange Interpolation (iota points)

c651fd

```

1Mint lagrange_iota(const vector<Mint> &f, const int c) {
2    const int n = int(size(f));
3    if (c < n) {
4        return f[c];
5    }
6    vector<Mint> fac(n), ifac(n);
7    fac[0] = 1;
8    for (int i = 1; i < n; i++) fac[i] = fac[i - 1] * i;
9    ifac[n - 1] = fac[n - 1].inv();
10   for (int i = n - 1; i >= 1; i--) ifac[i - 1] = ifac[i] *
11   i;
12   vector<Mint> g(n);
13   for (int i = 0; i < n; i++) {
14       g[i] = f[i] * ifac[i] * ifac[n - 1 - i];
15       if ((n - 1 - i) & 1) g[i] *= -1;
16   }
17   vector<Mint> lp(n + 1), rp(n + 1);
18   lp[0] = rp[n] = 1;
19   for (int i = 0; i < n; i++) lp[i + 1] = lp[i] * Mint(c -
20   i);
21   for (int i = n - 1; i >= 0; i--) rp[i] = rp[i + 1] *
22   Mint(c - i);
23   Mint ans = 0;
24   for (int i = 0; i < n; i++) ans += g[i] * lp[i] * rp[i +
25   1];
26   return ans;
27}

```

15 Linear Algebra**15.1 Gauss (mod p)**

7021b4

```

1// In-place RREF over  $\mathbb{Z}_{\text{mod}}$  (mod prime); returns {rank,
2// det}.
3// det is valid only when A is square.
4// Values must be normalized.
5pair<int, ll> gauss(vector<vector<ll>>& A) {
6    int n = A.size(), m = A[0].size();
7    int rank = 0; ll det = 1;
8    for (int c = 0; c < m && rank < n; c++) {
9        int piv = rank;
10       while (piv < n && !A[piv][c]) piv++;
11       if (piv == n) { det = 0; continue; }
12       if (piv != rank) {
13           swap(A[piv], A[rank]);
14           det = sub(0, det);
15       }
16       ll inv = Pow(A[rank][c], mod - 2);
17       det = mul(det, A[rank][c]);
18       for (ll& x : A[rank]) x = mul(x, inv);
19       for (int i = 0; i < n; i++) if (i != rank && A[i][c]) {
20           ll f = A[i][c];
21           for (int j = c; j < m; j++)
22               A[i][j] = sub(A[i][j], mul(f, A[rank][j]));
23       }
24       rank++;
25   }
26   return {rank, det};
27}

```

15.2 Solve Linear Equations

5a7bdd

```

1// Author: Gino
2// solve(Augmented Matrix), nullity == -1 if no solution
3struct Solution {
4    int nullity; vector<ll> xp; vector<vector<ll>> basis;
5};
6Solution solve(vector<vector<ll>>& A) { // augmented matrix
7    int n = A.size(), m = A[0].size() - 1;
8    auto [rank, det] = gauss(A);
9    for (int i = 0; i < n; i++) if (A[i][m] != 0) {
10        if (ranges::all_of(A[i] | views::take(m),
11                            [](ll x) { return x == 0; }))
12            return {-1, {}, {}};
13    }
14    Solution sol;
15    vector<int> pcol(rank); // pivot columns
16    vector<bool> isp(m, false); // is the column a pivot?
17    for (int i = 0; i < rank; i++) {
18        auto it = ranges::find(A[i], 1);
19        pcol[i] = ranges::distance(A[i].begin(), it);
20        isp[pcol[i]] = true;
21    }
22    sol.nullity = m - rank;
23    sol.xp.assign(m, 0);
24    for (int i = 0; i < rank; i++) sol.xp[pcol[i]] = A[i][m];
25    for (int j = 0; j < m; j++) if (!isp[j]) {
26        vector<ll> d(m, 0); d[j] = 1;
27        for (int i = 0; i < rank; i++) d[pcol[i]] = sub(0, A[i]
28        [j]);
29        sol.basis.push_back(d);
30    }
31    return sol;
32}

```

15.3 Gauss (GF(2) / XOR basis)

ace317

```

1// Solves  $Ax=b$  over GF(2), returning rank or -1
2// if inconsistent.
3// For an XOR basis, use all-zero b and read the nonzero
4// rows.
5const int N = 1000;
6using bs = bitset<N>;
7int gaussBinary(vector<bs>& A, vector<int>& b, bs& x, int m)
8{
9    int n = A.size(), rank = 0, br;
10   vector<int> col(m);
11   iota(col.begin(), col.end(), 0);
12   for (int i = 0; i < n; i++) {
13       for (br = i; br < n; br++)
14           if (A[br].any()) break;
15       if (br == n) {
16           for (int j = i; j < n; j++)
17               if (b[j]) return -1;
18           break;
19       }
20       int bc = (int)A[br]._Find_next(i - 1);
21       swap(A[i], A[br]);
22       swap(b[i], b[br]);
23       swap(col[i], col[bc]);
24       for (int j = 0; j < n; j++) if (A[j][i] != A[j][bc]) {
25           A[j].flip(i); A[j].flip(bc);
26       }
27       for (int j = i + 1; j < n; j++) if (A[j][i]) {
28           b[j] ^= b[i]; A[j] ^= A[i];
29       }
30       rank++;
31   }
32   x = bs(); for (int i = rank; i < n; i++) if (b[i]) {
33       x[col[i]] = 1;
34       for (int j = 0; j < i; j++) b[j] ^= A[j][i];
35   }
36   return rank;
37}

```

15.4 Solve Linear Equations (GF(2))

b481d7

```

1// solveBinary(A, b, m):  $Ax=b$  over GF(2),  $m$  = #variables.
2// nullity == -1 if no solution. Self-contained RREF (no
3// column swaps) so pivot columns are known for the basis.
4struct BinSolution { int nullity; bs xp; vector<bs>
5    basis; };
6BinSolution solveBinary(vector<bs> A, vector<int> b, int m)
7{
8    int n = A.size(), rank = 0;
9    vector<int> pcol; // pivot columns
10   for (int c = 0; c < m && rank < n; c++) {
11       int piv = rank;
12       while (piv < n && !A[piv][c]) piv++;
13       if (piv == n) continue;
14       swap(A[piv], A[rank]); swap(b[piv], b[rank]);
15       for (int i = 0; i < n; i++) if (i != rank && A[i][c])
16           A[i] ^= A[rank], b[i] ^= b[rank];
17   }
18   return {n - rank, xp, basis};
19}

```

```

15   pcol.push_back(c); rank++;
16 }
17 for (int i = rank; i < n; i++) if (b[i]) return {-1, {}},
18 {});
19 BinSolution sol{m - rank, {}, {}};
20 bs isp; // is the column a pivot?
21 for (int i = 0; i < rank; i++)
22   sol.xp[pcol[i]] = b[i], isp[pcol[i]] = 1;
23 for (int j = 0; j < m; j++) if (!isp[j]) {
24   bs d; d[j] = 1;
25   for (int i = 0; i < rank; i++) d[pcol[i]] = A[i][j];
26   sol.basis.push_back(d);
27 }
28 return sol;

```

15.5 Determinant

- Row swap: $-\det$; $k\vec{r}_i$: $k\det$; $\vec{r}_i + = k\vec{r}_j$: unchanged; $\det(A^T) = \det(A)$, $\det(AB) = \det(A)\det(B)$.
- Matrix-tree: number of spanning trees = any cofactor of Laplacian $D - A$. Directed (arborescences rooted at r): delete row/col r of $D_{in} - A$.
- Cayley: n^{n-2} labeled trees; forests of k rooted trees on given roots: kn^{n-k-1} .

16 Numerical

16.1 Integrate (Simpson / adaptive)

ff9766

```

1 // Simpson integration has fourth-order error
2 // in the step size.
3 template<class F>
4 double simpsonPart(double a, double b, F& f) {
5   double c = (a + b) / 2;
6   return (f(a) + 4 * f(c) + f(b)) * (b - a) / 6;
7 }
8
9 template<class F>
10 double simpson(double a, double b, F f, int n = 1000) {
11   double h = (b - a) / (2 * n);
12   double v = f(a) + f(b);
13   for (int i = 1; i < 2 * n; i++)
14     v += f(a + i*h) * (i&1 ? 4 : 2);
15   return v * h / 3;
16 }
17
18 // Adaptive Simpson integration to absolute error eps.
19 template<class F>
20 double adaptiveSimpson(F& f, double a, double b,
21   double eps, double whole) {
22   double c = (a + b) / 2;
23   double left = simpsonPart(a, c, f);
24   double right = simpsonPart(c, b, f);
25   double sum = left + right;
26   if (abs(sum - whole) <= 15 * eps ||
27     abs(b - a) < 1e-10)
28     return sum + (sum - whole) / 15;
29   return adaptiveSimpson(f, a, c, eps / 2, left) +
30     adaptiveSimpson(f, c, b, eps / 2, right);
31 }
32
33 template<class F>
34 double quad(double a, double b, F f, double eps = 1e-8) {
35   return adaptiveSimpson(f, a, b, eps, simpsonPart(a, b,
36   f));
37 }

```

16.2 Linear Programming

Standard form: $\max c^T x$ s.t. $Ax \leq b, x \geq 0$. Dual: $\min b^T y$ s.t. $A^T y \geq c, y \geq 0$; optima equal (strong duality). Complementary slackness: $\bar{x}_i, \bar{y}$ optimal iff for all i : $\bar{x}_i = 0$ or $\sum_j A_{ji} \bar{y}_j = c_i$, and for all j : $\bar{y}_j = 0$ or $\sum_i A_{ji} \bar{x}_i = b_j$. To standard form: minimize $\Rightarrow$ negate c ; $\geq$ row $\Rightarrow$ negate row; $=$ row $\Rightarrow$ both $\leq$ and $\geq$; free $x_i \Rightarrow x_i - x_i'$.

16.3 Simplex

8ad018

```

1 // Maximize c^T x subject to Ax <= b, x >= 0.
2 // Returns -inf if infeasible, inf if unbounded.
3 // Otherwise returns the maximum value and stores x.
4 // Numerical stability is not guaranteed.
5 using T = double;
6 using vd = vector<T>;
7 using vvd = vector<vd>;
8
9 const T eps = 1e-8;
10 const T inf = numeric_limits<T>::infinity();
11
12 struct LPSolver {
13   int m, n;
14   vi N, B;
15   vvd D;

```

```

17 LPSolver(const vvd& A, const vd& b, const vd& c)
18   : m(sz(b)), n(sz(c)), N(n + 1), B(m),
19     D(m + 2, vd(n + 2)) {}
20   for (int i = 0; i < m; i++)
21     for (int j = 0; j < n; j++) D[i][j] = A[i][j];
22   for (int i = 0; i < m; i++) {
23     B[i] = n + i;
24     D[i][n] = -1;
25     D[i][n + 1] = b[i];
26   }
27   for (int j = 0; j < n; j++) D[m][j] = -c[j];
28   N[n] = -1;
29   D[m + 1][n] = 1;
30 }
31
32 void pivot(int r, int s) {
33   T* a = D[r].data();
34   T inv = 1 / a[s];
35   for (int i = 0; i < m + 2; i++) if (i != r &&
36     abs(D[i][s]) > eps) {
37     T* b = D[i].data();
38     T inv2 = b[s] * inv;
39     for (int j = 0; j < n + 2; j++) b[j] -= a[j] * inv2;
40     b[s] = a[s] * inv2;
41   }
42   for (int j = 0; j < n + 2; j++)
43     if (j != s) D[r][j] *= inv;
44   for (int i = 0; i < m + 2; i++)
45     if (i != r) D[i][s] *= -inv;
46   D[r][s] = inv;
47   swap(B[r], N[s]);
48 }
49
50 bool simplex(int phase) {
51   int x = m + phase - 1;
52   for (;;) {
53     int s = -1;
54     for (int j = 0; j <= n; j++) {
55       if (N[j] == -phase) continue;
56       if (s == -1 || pair(D[x][j], N[j]) <
57         pair(D[x][s], N[s])) s = j;
58     }
59     if (D[x][s] >= -eps) return true;
60     int r = -1;
61     for (int i = 0; i < m; i++) {
62       if (D[i][s] <= eps) continue;
63       if (r == -1 || pair(D[i][n + 1] / D[i][s], B[i]) <
64         pair(D[r][n + 1] / D[r][s], B[r])) r = i;
65     }
66     if (r == -1) return false;
67     pivot(r, s);
68   }
69 }
70
71 T solve(vd& x) {
72   int r = 0;
73   for (int i = 1; i < m; i++)
74     if (D[i][n + 1] < D[r][n + 1]) r = i;
75   if (D[r][n + 1] < -eps) {
76     pivot(r, n);
77     if (!simplex(2) || D[m + 1][n + 1] < -eps) return -
78     inf;
79   }
80   for (int i = 0; i < m; i++) if (B[i] == -1) {
81     int s = 0;
82     for (int j = 1; j <= n; j++)
83       if (s == 0 || pair(D[i][j], N[j]) <
84         pair(D[i][s], N[s])) s = j;
85     pivot(i, s);
86   }
87   bool ok = simplex(1);
88   x = vd(n);
89   for (int i = 0; i < m; i++)
90     if (B[i] < n) x[B[i]] = D[i][n + 1];
91   return ok ? D[m][n + 1] : inf;
92 }

```

17 Combinatorics

17.1 Catalan Number

$$C_0 = 1, C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}, C_n = C_n^{2n} - C_{n-1}^{2n}$$

0	1	1	2	5
4	14	42	132	429
8	1430	4862	16796	58786

8	768	14	17280
---	-----	----	-------

- Let $\omega(n)$ = number of distinct prime factors.
- Minimum number with k distinct prime factors is the product of the first k primes.
 - $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 = 9699690$
 - $2 \cdot 3 \cdot \dots \cdot 23 = 223092870$
 - $2 \cdot 3 \cdot \dots \cdot 29 = 6469693230$
 - $n \leq 10^9 \Rightarrow \omega(n) \leq 9, n \leq 10^{18} \Rightarrow \omega(n) \leq 15$
- Number of square-free divisors = $2^{\omega(n)}$
 - $n \leq 10^9 \Rightarrow \leq 512, n \leq 10^{18} \Rightarrow \leq 32768$